What Do Language Models Need to Know About Their Tools?

Randeep Singh

Lucio AI `randeep@lucioai.com`

Abstract. Much of the text in a tool schema restates what the model can read elsewhere. We remove each piece of a tool's documentation in turn, then all of them, on the multi-turn tasks of the Berkeley Function Calling Leaderboard (BFCL), for four models. Only one removal costs every model: the parameter descriptions. The loss comes from argument construction, on a handful of parameters whose descriptions were the only place an allowed value, a format or an entity type was stated. An audit of where every constraint is stated predicts this. On τ^2-bench, where the same constraints live in `enum`s or in the conversation, the removal has no detectable cost. Writing BFCL's constraints into the schema returns every model to baseline with every parameter description blank; writing them into the tool description instead does so for two models, falls short for one and lands in between for the fourth. Removing every statement of what a tool does collapses tool selection on all three benchmarks tested.

Code and data: https://github.com/Eazydraft/exp-tool-semantics-ablation

1. Introduction

Function calling is the standard way to connect a language model to software. The model is shown a list of tools, each a JSON Schema (Wright et al., 2022) with a name, a description and, for every parameter, a name, a type and a description, and it answers a user by emitting calls against that list (Anthropic, 2024; OpenAI, 2023). Practitioners spend real effort on this text. They rewrite tool descriptions by hand, and a line of work studies how to write them well (Guo et al., 2026; Hsieh et al., 2023; Qu et al., 2025a; Yuan et al., 2025). What is missing is controlled evidence about which parts of that text a model relies on inside a realistic task, one with state to track, several turns to get through and a few dozen tools to choose from. Figure 1 gives our answer in one chart.

We use the multi-turn base section of BFCL (Mao et al., 2024; Patil et al., 2025), keep the tasks and the execution environment fixed, and vary only the tool documentation shown to the model. Four models from four providers ran every task under every condition, three of them with reasoning on and off. All comparisons

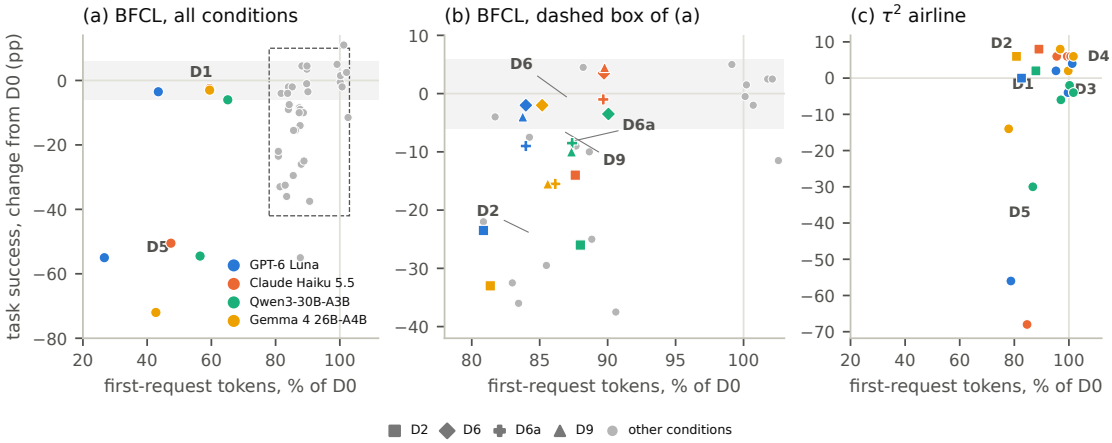


Figure 1 | What each piece of tool documentation costs to drop. Each point is one of four models under one condition on BFCL: the x-axis is the size of a task's first request, which carries the tool schemas, relative to the unmodified schema, and the y-axis is the change in task success. Removing tool descriptions (D1) cuts the schema by a third to a half at no detectable cost; removing parameter descriptions (D2) saves little and costs 14 to 33.5 points; writing the constraints those descriptions carried into the schema (D6) returns success to baseline at D2's cost; removing every piece of prose (D5) collapses success. Conditions are defined in Table 1. (b) enlarges the dashed box of (a), with the four conditions Section 6.3 discusses in colour and the rest in grey; the band is the widest run-to-run noise floor. (c) repeats D1 to D5 on τ^2-bench, where the constraints are stated structurally and nothing but D5 costs anything.

© 2026 Lucio AI. Licensed CC BY 4.0.

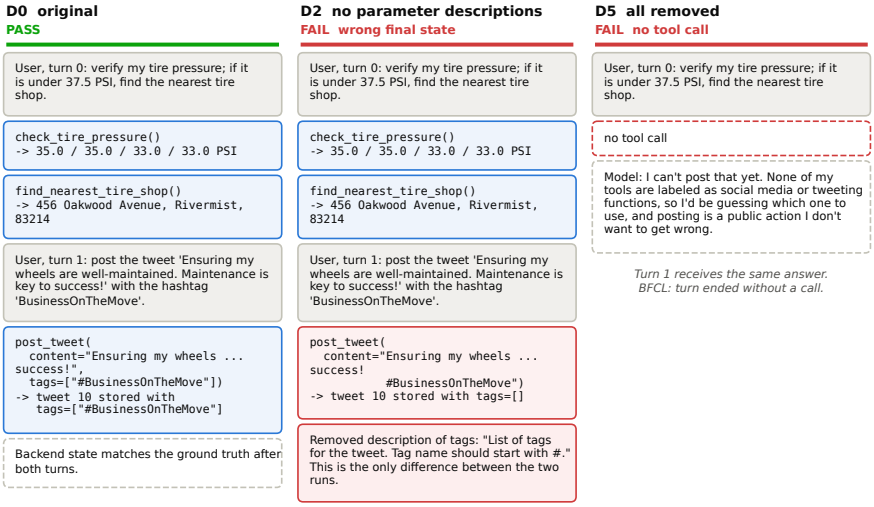


Figure 2 | One task under three conditions (Claude Haiku 5.5, BFCL task 78). With the original schema (D0) the hashtag goes in the `tags` list. Without parameter descriptions (D2) the first turn is identical, but the hashtag is folded into the tweet body and the final state no longer matches. With names and descriptions removed (D5) the model declines to guess which tool posts.

are paired at the task level, and a repeated baseline run measures run-to-run noise. Figure 2 shows what the ablation does to one task.

We then repeat the experiment on two other benchmarks. Apple's ToolSandbox (Lu et al., 2024) documents every tool redundantly, and the airline domain of τ^2-bench (Barres et al., 2025) states its constraints structurally. Together they test whether the BFCL result survives when the same information is stated somewhere else.

Section 3 fixes the vocabulary. A tool call involves two decisions, *selecting* a tool and *constructing* its arguments, and each needs a different kind of information: what a tool does, which values its parameters accept, and what values to supply. Any of this can be stated in several places, which we call *carriers*: the four text components, the structural fields of the schema, and the conversation itself. The experiment removes carriers one at a time, and then all at once, to see how the information they carry affects tool use. We find:

- **Where a constraint is stated predicts what its removal costs.** Removing parameter descriptions (D2) is the one single removal that hurts every model on BFCL, by 14 to 33.5 percentage points (Section 5.1). The damage is at argument construction rather than tool selection (Section 6.1), and it is concentrated on six parameters whose description alone stated an allowed value, a format or an entity type (Section 6.2). An audit of every ground-truth argument turns this into a property of the interface rather than of the models. BFCL keeps two fifths of its constraints in prose and nowhere else; τ^2-bench states the same kind of constraint as an `enum` or supplies the value in the conversation, and there the same removal has no detectable cost (Sections 5.2 and 5.3). ToolSandbox's descriptions hold no constraint at all, so its null speaks only to the next point.
- **Tool-purpose information is stated redundantly.** Removing tool descriptions, tool names or parameter names one at a time leaves success within each model's own run-to-run noise for all but one model and condition (Sections 5.1 and 5.3). Removing all of them collapses tool selection on all three benchmarks (Section 5.4). On τ^2 airline the models then cancel the reservations they meant to look up, because the cancel and look-up tools share a signature and nothing else is left to tell them apart.
- **Structure can carry the constraints instead.** Writing the BFCL constraints into the schema as `enum`s, `pattern`s and two more informative parameter names (D6) returns every model to its baseline with every description blank (Section 6.3). Stating the same constraints as sentences in the tool description (D9) recovers most of the loss for every model and all of it for two; Gemma 4 stays 15.5 points short and Qwen3 8 to 10, so the information has to be stated somewhere the model reads, and the schema was the place every model read. The carriers are not freely interchangeable, though. Qwen3 loses 11.5 points when parameter names become generic although every description stays, and removing parameter names and descriptions together (D8) costs more than removing either.

2. Related work

Tool use has progressed from inserted API calls (Schick et al., 2023) and ReAct prompting (Yao et al., 2023) to models trained on tens of thousands of APIs (Patil et al., 2023; Qin et al., 2024). Qu et al. (2025b) decompose tool use into planning, selection, calling and response generation, which is the framing behind Section 3.3. Benchmarks differ in what they score. API-Bank (Li et al., 2023) and BFCL's single-turn categories (Yan et al., 2024) score individual calls, whereas BFCL V3 (Mao et al., 2024), τ-bench (Yao et al., 2025) and ToolSandbox (Lu et al., 2024) score a whole trajectory by its outcome.

Prior work treats documentation as one object to improve, whether by showing that it can replace demonstrations (Hsieh et al., 2023) or by rewriting it (Guo et al., 2026; Qu et al., 2025a; Yuan et al., 2025). We ask instead which information in it is used and where that information has to be. The closest study is Clouâtre (2026), who found, for two models, no measurable loss from moving parameter detail out of the tool description into per-parameter fields; our D1 result shows the same redundancy. Robustness studies perturb the query, the toolkit, the interface or the outputs and find consistent degradation (Rabinovich and Anaby-Tavor, 2025; Xu et al., 2026; Zheng et al., 2026), and Chattopadhyay et al. (2026) find description and schema edits less harmful than distractor tools with overlapping descriptions, which matches our D1 and D3 nulls. None of these removes a component outright while holding the benchmark fixed. On the form of the interface, Sigdel and Baral (2026) find that schema conditions reduce interface misuse but not semantic misuse, the design closest to our D6, and Johnson et al. (2025) find that replacing JSON tool calls with natural-language tool selection raises accuracy.

3. Background

This section fixes the terms the rest of the paper uses: the interface a model is shown (Section 3.1), the interaction loop and how stateful benchmarks score it (Section 3.2), and the two decisions a model makes at every step together with the information each one needs (Section 3.3).

3.1. The function-calling interface

Provider-native function calling (OpenAI, 2023) and the Model Context Protocol (Anthropic, 2024) share one format. Each tool is a JSON object with a `name`, a `description` and a `parameters` JSON Schema (Wright et al., 2022), and every model in this study receives exactly that serialised schema. Formally, a tool is a tuple $\tau = (n, d, P)$ of a name, a description and a parameter specification. Each $p \in P$ has a name, a type, a description and *structural fields*, the ones JSON Schema defines with a fixed meaning: `required`, `default`, an `enum` of admissible values, a `pattern` a string must match, and nested specifications for object-typed parameters. The model is shown a tool set $\mathcal{T} = \{\tau_1, \ldots, \tau_K\}$ and answers with text or with calls (n, a), where a should conform to P.

The structural fields are machine-readable and have one meaning each. The other four pieces of text, which we call the *prose carriers*, are the tool name, the tool description, the parameter names and the parameter descriptions; we count names as prose because each is free text with no fixed meaning. A description may say anything, including what a structural field could have said. "Type of order (Buy/Sell)" states an admissible-value set, and "Tags should start with #" states a format. We call a constraint on an argument value *structural* if a structural field expresses it and *prose-only* if only a description states it. More generally, a *carrier* is any place a piece of information is stated: the four prose carriers, the structural fields, and the conversation, meaning the user's request and the outputs of earlier calls.

3.2. The interaction loop and its evaluation

A tool-calling conversation alternates between a user and the model. Within a user *turn* the model takes one or more *steps*: it is sent the conversation so far and the tool set, and it replies with calls or with text. Each call is executed against a backend, whose result or error is appended before the next step, and a text reply ends the turn. A *stateful* backend lets a call change what later calls and turns see. The benchmarks used here score outcomes rather than call sequences. BFCL compares the backend state after each turn with the state the ground-truth calls produce (Mao et al., 2024); τ-bench (Yao et al., 2025) and ToolSandbox (Lu et al., 2024) simulate the user with a language model and score the final state or the similarity to milestones. Outcome scoring is what makes a documentation ablation clean: the model may reach the required state by any sequence of calls, while the backend, the user and the ground truth stay identical across conditions.

3.3. Two decisions and what they need

At every step the model makes two decisions: it *selects* a tool $n \in \mathcal{T}$ and *constructs* an argument object a for it. Three kinds of information feed these decisions. *Tool purpose*, what each tool does, is what selection needs. *Argument constraints*, which values a parameter admits, in what format, and what kind of entity it names, are what construction needs beyond the parameter's type. *Argument values* are the values themselves. They come from the user's request, from earlier results or from convention; they are not documentation, and a value copied from the conversation needs no constraint to be stated.

Each piece of information may be stated in several carriers at once. A tool's purpose is usually stated by its name, by its description and by its parameter list; an admissible set may be stated by an `enum` and again by a description. Where two carriers state the same thing, removing one leaves the other, and nothing should change. Where one carrier is the only statement, removing it removes the information, and the model must then infer it, discover it by interacting with the backend, or fail. This is the prediction the experiment tests: removing a carrier should cost nothing if what it carried is stated elsewhere or is not needed, and should cost at the decision that needs the content if it was the sole statement. In the trajectories, a *selection failure* shows as the right tool never being called, a *construction failure* as the right tool being called with wrong values, and recovery as a later call that corrects an earlier one.

4. Experimental setup

4.1. Benchmarks

The three benchmarks were chosen for where they state the information a call needs. BFCL keeps many argument constraints in parameter descriptions and nowhere else. ToolSandbox documents every tool redundantly, and its descriptions hold no argument constraint, so it can test only the redundancy of tool-purpose information. τ^2-bench states its constraints as `enum`s or supplies the values through the conversation. Running the same ablation on all three therefore tests the same information under different placements, which is what the audit of Section 5.2 makes precise.

BFCL multi-turn base. The `multi_turn_base` category (Mao et al., 2024) has 200 tasks of one to seven user turns (median four) over eight stateful Python backends. A task involves one or two backends, and the model is shown every function of those backends, 18 to 39 tools (median 28). A turn passes if the backend state matches the state the ground-truth calls produce and any required outputs were produced; a task passes only if every turn passes; a turn may take at most 20 steps. We used BFCL at commit `6ea57973` (Gorilla LLM contributors, 2026), unmodified, at its default temperature of 0.001.

ToolSandbox. ToolSandbox (Lu et al., 2024) (commit `c8571d7`) simulates the user with GPT-4o-2024-05-13, as in the original paper. Its tools are Python functions over a phone-like world, and their schemas are generated from docstrings, so every description is one short line. The score is a similarity in $[0, 1]$ to the scenario's milestones. We run the 91 base scenarios that need no paid web API, each with 24 to 28 tools (Appendix E).

τ^2-bench airline. The airline domain of τ^2-bench (Barres et al., 2025) (version 2.0, commit `19f13ea`), the successor of τ-bench (Yao et al., 2025), has 50 tasks in which an agent shown 14 tools and the airline policy serves a user simulated by GPT-4.1, the benchmark's default. A task is rewarded 1 only if the final database state matches the state the ground-truth actions produce and, for the six tasks that require it, the agent's replies pass the benchmark's judge. Three of its parameters carry `enum`s, and the values it needs are identifiers, codes and dates that the user states or a look-up returns, so its constraints are stated outside the prose (Appendix F).

4.2. Documentation conditions

Each condition is an operation on the tuple (n, d, P) that leaves the structural fields untouched (Table 1). D1 to D4 each remove one prose carrier, so each measures what that carrier stated that no other carrier did. D1 removes the tool description and, with it, the prose inside BFCL's response schema, which documents what the tool returns; the structure of that schema stays. D5 removes all four prose carriers. Even so, it removes most of the semantics rather than all of it: types, requiredness, enums, outputs and the user's request remain, and tools stay distinguishable by argument structure wherever that structure differs. When names

Table 1 | Documentation conditions. Generic tool names are `tool_01`...`tool_NN` in listing order; generic parameter names are `arg_1`...`arg_n` per tool, recursively. Structural fields are kept throughout. †D1 also blanks the response-schema descriptions. ‡D6 and D6b rename `cityA`/`cityB` to `zipcodeA`/`zipcodeB`. §D9 appends the D6 constraints to the tool description as sentences.

	Tool names	Tool descriptions	Parameter names	Parameter descriptions
D0	original	original	original	original
D1	original	**removed**†	original	original
D2	original	original	original	**removed**
D3	**generic**	original	original	original
D4	original	original	**generic**	original
D5	**generic**	**removed**	**generic**	**removed**
D6	original	original	original‡	**removed**; constraints in schema
D6a	original	original	original	**removed**; enums and patterns in schema
D6b	original	original	original‡	**removed**
D7	original	original	original	**removed**; six kept (Table 10)
D8	original	original	**generic**	**removed**
D9	original	original§	original	**removed**

are generic, the model's calls are translated back to the original names before the benchmark executes and scores them, so the backend and the evaluator see exactly what they would in D0. This holds on all three benchmarks. The conditions after D5 blank every parameter description, as D2 does, and test the explanation that Section 6.2 arrives at. D6 writes every prose-only constraint of the BFCL schemas into a structural field (an `enum` for 11 parameters, an item `pattern` for two, and new names for `estimate_distance.cityA` and `cityB`); D6a and D6b split those into the structural fields alone and the renames alone; D9 appends the same constraints as sentences to the tool description instead; D7 keeps only the six descriptions of Table 10; D8 removes parameter names and parameter descriptions together while keeping tool names and descriptions. The exact transformations are in Appendix B. The conditions also change how much the model reads: the first request of a task, which carries the schemas, is 2,585 to 5,707 tokens at D0 depending on the model's tokenizer, and D1 cuts it by 35 to 57%, D2 by 12 to 19% and D5 by 44 to 73% (four default configurations).

4.3. Models and harness

We evaluated GPT-6 Luna (OpenAI, 2026), Claude Haiku 5.5 (Anthropic, 2026), Qwen3-30B-A3B-Instruct-2507 (Yang et al., 2025) and Gemma 4 26B-A4B (Google DeepMind, 2026) through OpenRouter (OpenRouter, 2026), all in native function-calling mode. On BFCL, Luna and Haiku reason by default and were also run with reasoning off; Gemma 4 does not reason by default and was also run with reasoning on; Qwen3 has no reasoning switch. That gives seven configurations. Each ran all 200 tasks under D0 to D5, and each model's default setting also ran the conditions after D5. ToolSandbox and τ^2 were run under D0 to D5 in each model's default setting. Each configuration's D0 to D5 were run together, so every ablation-versus-D0 comparison shares a batch; the conditions after D5 were run separately, together with a second D0 run that Appendix B uses as their baseline. The harness subclasses each benchmark's OpenAI-compatible agent and differs from a stock run only in routing to OpenRouter, applying the condition, translating generic names back, and recording every trajectory in full (11 200 BFCL trajectories, 2 184 ToolSandbox plays and 2 400 τ^2 simulations). Four Qwen3 BFCL trajectories ended in a provider error and count as failures.

4.4. Metrics and statistics

Task success is each benchmark's own verdict. The behavioural metrics are statistics of the loop of Section 3.2, computed from the trajectories without any model-based judging. A *call* is a tool call the benchmark decoded, and a *step* is one model request. A call is *on-path* if its tool appears in the ground truth, *off-path* if not, and *failed* if the backend returned an error. For each ground-truth call in turn t, a *tool hit* means the model called that tool in any step of turns 0 to t, and an *exact hit* means one of those calls also had identical arguments (with no case folding, so "`buy`" does not match "`Buy`"). *Tool-selection accuracy* is tool hits over ground-truth calls, and *argument accuracy given the right tool* is exact hits over tool hits. For each failed task, the *first divergence* is the earliest ground-truth call without an exact hit; it counts as a construction failure if the tool was called with other arguments and as a selection failure if the tool was never called. On τ^2 the ground truth is the task's action list; on ToolSandbox it is the earliest milestone below similarity 1 that needs a tool. *Recovery rate* is the share of tasks with at least one failed call that still passed. These metrics measure agreement with the ground-truth calls, which is not the same as benchmark correctness. A task can pass by

Table 2 | BFCL task success (%) per condition, 200 tasks per cell, with the paired change versus D0 in percentage points. Bold, starred cells are significant at the Bonferroni threshold $p < 0.0014$ (exact McNemar test; discordant cells, intervals and p-values in Table 5).

Model	D0	D1	D2	D3	D4	D5
GPT-6 Luna, reasoning	56.0	52.5 (−3.5)	**32.5** (−23.5)*	57.5 (+1.5)	58.5 (+2.5)	**1.0** (−55.0)*
GPT-6 Luna, no reasoning	54.0	48.5 (−5.5)	**32.0** (−22.0)*	59.5 (+5.5)	56.0 (+2.0)	**0.0** (−54.0)*
Claude Haiku 5.5, reasoning	50.5	48.0 (−2.5)	**36.5** (−14.0)*	55.5 (+5.0)	**61.5** (+11.0)*	**0.0** (−50.5)*
Claude Haiku 5.5, no reasoning	47.0	42.0 (−5.0)	**27.5** (−19.5)*	47.5 (+0.5)	46.5 (−0.5)	**0.0** (−47.0)*
Qwen3-30B-A3B, no reasoning	54.5	48.5 (−6.0)	**28.5** (−26.0)*	52.5 (−2.0)	**43.0** (−11.5)*	**0.0** (−54.5)*
Gemma 4 26B-A4B, no reasoning	72.0	69.0 (−3.0)	**39.0** (−33.0)*	71.5 (−0.5)	74.5 (+2.5)	**0.0** (−72.0)*
Gemma 4 26B-A4B, reasoning	76.0	70.5 (−5.5)	**42.5** (−33.5)*	78.0 (+2.0)	76.0 (+0.0)	**2.5** (−73.5)*

a different but valid sequence of calls, and a call that agrees with the ground truth can still sit in a failed task. They are diagnostic proxies for the selection and construction decisions of Section 3.3, and the first-call versions are the cleanest, because nothing has yet gone wrong when the first call is made.

Information audit. For every argument of every ground-truth call, we record where the constraint on its value is stated. A parameter description *states a constraint* if it gives an admissible-value list, a format, unit or range, or an entity type that the parameter name does not give (zip codes for `cityA`). A single example ("such as ZFA04Y") or a restatement of the name does not count. The judgement is made per parameter and listed in Appendix C. A structural field states a constraint if the parameter has an `enum` or `pattern` or is a boolean. A value is *copied* if it occurs verbatim in the user's messages up to that turn or in the output of an earlier call, with outputs taken from a trajectory that passed the task at D0. ToolSandbox's relative-date reminders ("tomorrow 5PM") are the one exception: the benchmark computes their expected timestamp from the clock at evaluation time, so the audit checks the timestamp the model passed in that trajectory. Every argument then falls into one of five classes: description only; description and structure; structure only; none, copied; none, constructed. Separately, a tool's purpose counts as stated by its name, by its description, and by its signature when no tool offered alongside it has the same parameter names and types.

Every task appears in every condition, so each ablation is compared with D0 by an exact two-sided McNemar test (McNemar, 1947) on the discordant cells, with a 95% interval for the paired difference from a task-level bootstrap of 2 000 resamples (Efron, 1979). With 35 BFCL comparisons (five ablations × seven configurations) the Bonferroni threshold is $p < 0.0014$ (Dunn, 1961); the 20 τ^2 comparisons give $p < 0.0025$. The conditions after D5 form a third family of 52 comparisons (Appendix B); their p-values are reported uncorrected, and a result is called significant there only if it also passes that family's threshold of $p < 0.001$. A null result is reported with its *equivalence bound*, the largest change in either direction that the interval allows.

5. Results

5.1. BFCL: constraint information with a single carrier

Removing parameter descriptions is the only single removal that hurts every model. Every configuration in Table 2 and Figure 3 has the same shape: a small dip at D1 (tool descriptions removed), a large drop at D2 (parameter descriptions removed), no change at D3 (generic tool names) and a collapse at D5 (all four removed). D2 and D5 are significant for all seven configurations at the corrected threshold, and the D2 discordant cells are lopsided. In each configuration, between 42 and 75 tasks pass in D0 but fail in D2, while only 5 to 16 pass in D2 but fail in D0 (Table 5). D1 and D3 look different: their discordant cells are of similar size in both directions, and every interval includes zero, although all seven D1 differences are negative (−2.5 to −6.0 pp). D4 (generic parameter names) is harmless for Luna and Gemma 4. It costs Qwen3 11.5 pp ($p = 0.001$) even though every description Qwen3 could read stayed in place, and it shows an apparent +11.0 pp gain for Haiku with reasoning that is absent with reasoning off.

Two identical runs differ by about as much as D1 or D3. Claude Haiku 5.5's second D0 run passed 56.5% of tasks against 50.5% in the first: 9 tasks passed only in the first run and 21 only in the second ($\Delta = +6.0$ pp, $p = 0.0428$; Table 16). This gap is the floor against which single-condition differences have to be read; a second D0 run for each of the other three default configurations gives floors of the same order, 0.5 to 6.0 pp in absolute value across the four (Appendix B.4). D3 sits at the floor, and so does D4 for three of the four models; both are null results. D1 is a possible small cost. Haiku's D1 gap widens to −8.5 pp ($p = 0.008$) when measured against the second run, and D1 is negative for all seven configurations. The reasoning pairs of one model are not independent, so the honest sign test is over the four models, where four negatives give

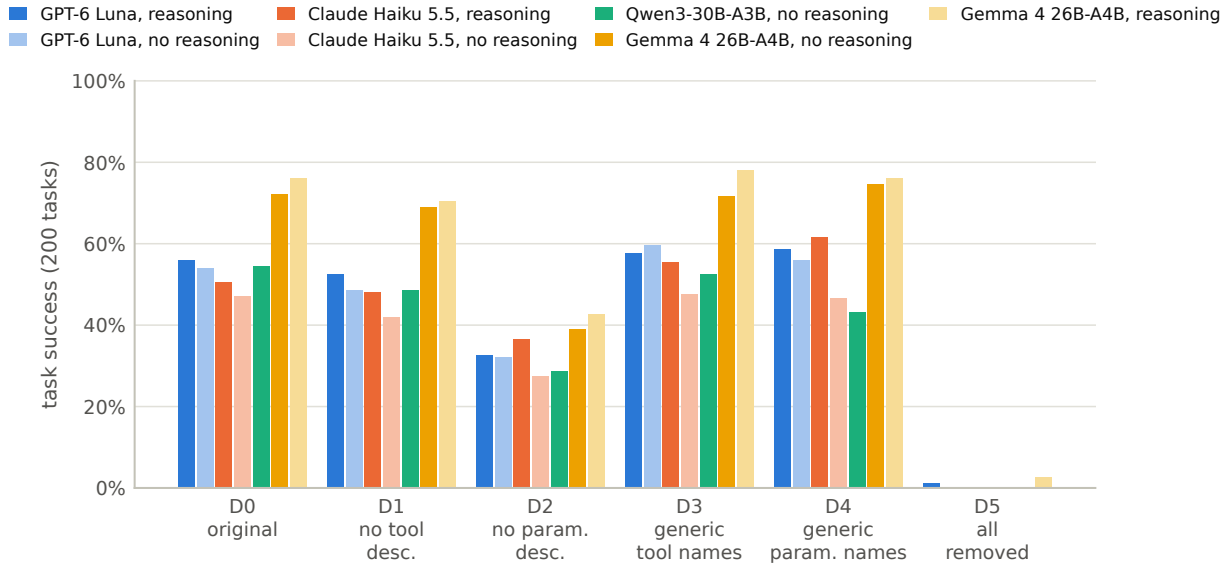


Figure 3 | Task success rate by condition for the seven configurations (lighter tint: the non-default reasoning setting). Each bar is 200 tasks; bars within a configuration are paired.

Table 3 | Information audit. For every argument of every ground-truth call: where the constraint on its value is stated (the five classes of Section 4.4), how many of the arguments with a description constraint nonetheless have their value verbatim in the conversation, the kinds of constraint the descriptions state, and the number of parameters whose description is the only statement of a constraint (listed in Appendix C).

				where the constraint on the value is stated					of the description-constrained arguments				
Benchmark	calls	args	params	desc. only	desc. + struct.	struct. only	none, copied	none, constructed	value verbatim	enum	format	entity	desc.-only params
BFCL multi-turn	1142	1954	132	772	0	56	1029	97	495	217	496	59	41
ToolSandbox	172	176	24	14	0	32	108	22	14	0	14	0	2
τ^2-bench airline	142	360	29	81	20	30	216	13	95	20	81	0	6

$p = 0.125$; the pattern is suggestive, no more. The sizes, -2.5 to -6.0 pp, sit inside the floor, so one run per condition cannot resolve them. D2 and D5, by contrast, are an order of magnitude above the floor and significant against either run. Reasoning shifts baseline success but leaves the D2 drop in place. The three pairs that differ only in the reasoning switch show D2 drops of -23.5 and -22.0 pp (Luna with reasoning on and off), -14.0 and -19.5 pp (Haiku) and -33.0 and -33.5 pp (Gemma 4), even though reasoning budgets respond to the interface (Table 6).

5.2. Where the information is stated: an audit of the ground truth

On BFCL two fifths of the ground-truth arguments have a constraint stated only in a description; on the other two benchmarks the constraints are structural or the values are copied. Table 3 classifies the 1954 arguments of BFCL's 1 142 ground-truth calls, the 176 values that ToolSandbox's milestones pin, and the 360 arguments of τ^2's 142 ground-truth actions. The three benchmarks differ sharply. On BFCL, 772 arguments (39.5%), spread over 41 parameters, have a constraint whose only statement is the description: allowed-value lists written as prose, formats, units and ranges, and entity types that the parameter name hides. Half of all BFCL arguments (53%) are copied from the conversation and need no constraint at all, and so are 495 of the 772 description-constrained ones; that leaves 277 arguments for which a description is the only route to the value. ToolSandbox has 14 description-only arguments (8.0%), all of them the POSIX-timestamp format of `reminder_timestamp`, whose value a conversion tool returns; a further 32 of its values are booleans, and 61% are copied. τ^2 has 81 description-only arguments (22.5%), airport-code and date formats, but 95 of its 101 description-constrained values appear verbatim in the user's messages or in an earlier look-up, its cabin and flight-type lists are stated both in the description and in an `enum`, and 60% of its values are copied. Tool purpose, by contrast, is stated by name and description for every tool on every benchmark, and additionally by a signature no other tool shares for 70% of BFCL's ground-truth tool instances, 45% of ToolSandbox's and 53% of τ^2's (Table 17). Section 6.2 returns to the BFCL parameters behind these counts.

5.3. ToolSandbox and τ^2-bench: no detectable cost where the information is stated elsewhere

Removing parameter descriptions has no detectable cost on either benchmark, for a different reason on each. On ToolSandbox, D2 (parameter descriptions removed) changes mean similarity by −6.3 to +2.0 points depending on the model (sign-flip $p \geq 0.07$) and leaves the share of exactly solved scenarios unchanged for every model (Appendix E). The audit says why. Of its 176 pinned values, the 14 whose only constraint is a description are timestamps that an earlier conversion call returns, and in the passing D0 trajectories all 14 of them are that call's output. The format is enforced by the tool that produces the value, and the model only has to copy it, so the information D2 removes was never needed. On τ^2 the information was needed, but it was stated elsewhere. D2 changes success by +0.0 to +8.0 pp ($p \geq 0.34$; Table 25). With 50 tasks per condition, the single-trial intervals bound the change at 12, 20, 16 and 22 pp for Luna, Haiku, Qwen3 and Gemma 4; running D0 and D2 for four trials per model narrows this to a trial-averaged change of −4.5 to +5.0 pp with bounds of 8.0 to 10.5 pp (Appendix F), which is below the smallest D2 effect seen on BFCL. Cabin and flight type keep their `enum`s, and 95 of the 101 description-constrained values, airport codes and dates, appear verbatim in the user's messages or in the reservation the agent has just looked up. The two nulls therefore say different things. ToolSandbox's descriptions hold no argument constraint, so its D2 null shows only that tool-purpose information is stated redundantly; τ^2 shows one constraint surviving in an `enum` when its prose statement goes. D1, D3 and D4 are likewise nulls on both benchmarks. On τ^2 every change is within −6.0 to +8.0 pp with bounds of 10 to 22 pp (Table 26); on ToolSandbox every change is within a few points, except for Gemma 4's −6.0 at D3 ($p = 0.003$). Qwen3's BFCL D4 cost does not reappear on τ^2 (−4.0 pp, bound 16).

5.4. Removing every carrier of a tool's purpose collapses selection on all three benchmarks

D5 (tool names, tool descriptions, parameter names and parameter descriptions all removed) is significant for every BFCL configuration (−47.0 to −73.5 pp), for every ToolSandbox model (−39 to −49 similarity points, $p < 0.0001$) and for three of the four models on τ^2 (−30 to −68 pp; Gemma 4's −14.0 has $p = 0.12$). These are failures of selection. On BFCL, tool-selection accuracy falls to 16 to 60%, and most failed tasks begin with a ground-truth tool that is never called (Table 9), whereas at D2 most begin with the right tool called with wrong arguments. On ToolSandbox, among the 75 to 78 plays per model below similarity 1, the earliest unmet milestone needs a tool that was never called in 63 to 70 of them, against 5 to 9 at D0 (Table 24). On τ^2, 15 to 39 of the 36 to 50 failed tasks per model begin with an action's tool never called, and nearly all of them contain a state-changing call that the ground truth does not (Table 29). The mechanism is visible in the calls. `cancel_reservation` and `get_reservation_details` both take a single `reservation_id`, so once names and descriptions are gone nothing in the schema tells them apart, and every model's look-ups turn into cancellations (Appendix F). Tool-purpose information is stated redundantly enough to survive any one removal; D5 removes every statement of it at once.

6. Analysis

Section 3.3 named three places where a missing parameter description could hurt: at selection, at construction, or in the model's attempts to recover by interacting. The BFCL trajectories let us tell the three apart.

6.1. Selection versus construction

D2 leaves selection intact and damages construction. Between D0 and D2 (parameter descriptions removed), tool-selection accuracy moves by only 2.6 to 8.8 points, and part of that is cascade: a turn derailed by a wrong argument may never reach its later tools. Argument accuracy given the right tool, by contrast, falls for every configuration, from 90.7–94.7% to 79.5–86.3% (Figure 5a,b). The first call of each task is free of cascade (Figure 5c), and it tells the same story. In D2 the right tool is chosen first at least as often as in D0 for every configuration, but the share of first calls with the right tool and the right arguments falls by 11.0 to 16.0 points. The construction failure is therefore already present on the first call of a task, before any state has accumulated; the multi-turn setting is where the study is run, not the source of the effect. The first divergence of failed tasks agrees (Figure 6): relative to D0, construction failures rise by 34 to 66 per configuration and selection failures by only 2 to 11. D5 fails at selection instead (Section 5.4).

Table 4 | The conditions that move the constraints, each model in its default reasoning setting: task success (%) with the change versus D0 in parentheses. All blank every parameter description. D6a adds `enum`s and patterns, D6b renames `cityA`/`cityB`, D6 does both, D9 states the same constraints as sentences in the tool description. Paired tests in Appendix B.

Model	D0	D2	D6a	D6b	D6	D9
GPT-6 Luna	56.0	32.5 (−23.5)	47.0 (−9.0)	34.0 (−22.0)	54.0 (−2.0)	52.0 (−4.0)
Claude Haiku 5.5	50.5	36.5 (−14.0)	49.5 (−1.0)	41.5 (−9.0)	54.0 (+3.5)	55.0 (+4.5)
Qwen3-30B-A3B	54.5	28.5 (−26.0)	46.0 (−8.5)	25.0 (−29.5)	51.0 (−3.5)	44.5 (−10.0)
Gemma 4 26B-A4B	72.0	39.0 (−33.0)	56.5 (−15.5)	36.0 (−36.0)	70.0 (−2.0)	56.5 (−15.5)

6.2. Which descriptions matter

Six parameters carry the effect, and each description was the only statement of a constraint. Table 10 links each new argument error under D2 to the description that was removed. The same six parameters lead for every configuration: `order_type`, `ignitionMode`, `cityA`, `cityB`, `tags` and `mentions`. Each of their descriptions held a prose-only constraint in the sense of Section 3.1: an allowed-value list ("Buy/Sell" for `order_type`, "START, STOP" for `ignitionMode`), an entity type the name hides (`cityA` and `cityB` take zip codes, not city names), or a format (`tags` start with `#`, `mentions` with `@`). Without the description, models wrote "`buy`" for "`Buy`", passed city names where zip codes were expected, or folded the hashtag into the tweet body. The counts nest as follows. The audit lists 41 description-only parameters; D6 rewrites the 13 of them that hold an enum or a format, plus two renames; six of the 13 carry nearly all of the D2 loss, and all 153 of their ground-truth arguments are description-only. The other description-only parameters, led by date and pedal-position formats, go wrong far less often (Table 10). At task level the effect sits on the tasks that use these parameters. The ground truth of 91 of the 200 tasks uses at least one of the six, and 63 to 91% of the tasks that pass in D0 and fail in D2 are among those 91. On the 91 tasks the D2 drop is 30.8 to 73.6 pp. On the other 109 tasks it is between −10.1 and +4.6 pp, within the noise floor for six of the seven configurations (Table 11). Not all of the loss is substantive. Of the tasks that pass in D0 and fail in D2, 11 to 50% per configuration begin with a value that differs from the ground truth only in case ("buy" for "Buy", "start" for "START"; per configuration in Table 11, with Haiku with reasoning the one at half). The rest begin with a different value, such as a city name for a zip code, or with a tool never called. Case normalisation in the checker would therefore recover at most that share. Parameter descriptions matter here less as explanations of meaning than as the place where this benchmark happens to keep its enums, formats and entity conventions. Section 6.3 asks whether a structural field can hold them instead.

6.3. Restoring the structure without the prose

Structure carries the constraints, and names and descriptions restate each other. The conditions after D5 test the explanation of Section 6.2 directly (Table 4 and Figure 1; Appendix B reports them in full). Figure 1 puts every condition on one chart, the size of the schema against what it costs: D1 is the biggest saving at no detectable cost, D5 is the cheapest and collapses, and D6 sits near D2's cost at D0's success. Two caveats apply to the chart: the first request is what providers cache, so the saving is context budget more than money, and D1 also removes the prose in BFCL's response schemas, so part of its saving is not a tool-description saving in the usual sense. D6 writes the prose-only constraints into the schema and blanks every description. It lands within 3.5 points of D0 for every model, with no significant difference and equivalence bounds of 6.5 to 10.0 pp, and every D6–D2 difference is significant, recovering 17.5 to 31.0 points. D7, which keeps only the six descriptions, recovers most but not all of the loss. Splitting D6 into its two moves shows that the structural fields carry most of the recovery (D6a, +13.0 to +17.5 pp over D2) and that the two renames add the rest only in combination with them, because the tasks that need the zip codes also need the enums (Appendix B.2). Stating the same constraints as sentences in the tool description instead (D9) recovers about as much as the fields alone. It brings Luna and Haiku back to baseline. Gemma 4 stays 15.5 pp below it against its first D0 run and −16.5 pp against a second one made alongside D9 ($p < 0.0001$), and Qwen3 lands in between, 8 to 10 pp short on both baselines with intervals that exclude zero but a difference that does not pass the corrected threshold: the constraint has to be stated somewhere the model reads, and the schema was the place every model read. D8 removes parameter names and descriptions together and costs 9.0 to 22.0 pp more than D2 (Appendix B.3): what a parameter is was stated twice, and removing both statements leaves only the type.

6.4. Behaviour, recovery and failure outcomes

The third possibility of Section 3.3 is *interactive recovery*: a model that lacks a piece of information discovers it by calling a tool, reading the error, and calling again. Under the single ablations this does not happen. D2 adds only 0.1 to 2.8 calls per task over D0, and what changes is the composition of the calls rather than their number: failed calls per task rise from at most 0.03 to between 0.10 and 0.51 (Appendix D). Recovery after an execution error is poor in D2, at 5 to 22% of the 19 to 30 tasks that contained one, and all but absent in D5, where six configurations recovered none of their 17 to 102 such tasks. Under D5 two opposite behaviours appear. Haiku and Luna with reasoning end the first turn without any call in 22 to 35% of tasks, while Qwen3, and Gemma 4 with reasoning, keep probing until the step cap. Neither behaviour passes tasks (case studies in Appendix G).

7. Discussion

For tool designers. The audit of Section 5.2 doubles as a design check. For every parameter description, ask whether it states a constraint that the structural fields do not. If it does, state the constraint there as well, and keep the prose for what structure cannot say. The token numbers of Section 4 give the other side of the trade: dropping tool descriptions cut the first request by 35 to 57% at a cost inside the noise floor, whereas dropping parameter descriptions cut it by 12 to 19% and cost 14 to 33.5 pp on BFCL and nothing detectable on τ^2; what a description may be cut to depends on what else states its constraints (Figure 1). Qwen3's D4 result (generic parameter names) is the caution: the information was still in the descriptions, yet replacing the names with `arg_N` cost 11.5 pp, so carriers are not interchangeable for every model.

Why tool names and descriptions could be removed one at a time. A tool's purpose is stated by its name, by its description and, for most tools, by a signature no other tool shares. Removing any one of these leaves the others, and D5 shows the information was used, because removing all of them collapses selection on every benchmark. The design cannot rank names, descriptions and parameter names against each other, since each is one redundant carrier of the same information. It does show where the redundancy ends. τ^2's cancel and look-up tools, and ToolSandbox's four `set_*_status` tools, share a signature, and once names and descriptions are gone the models act on the wrong one. On τ^2 the wrong one was destructive: models cancelled reservations they meant to look up rather than ask. We note this as an observation rather than a safety result, since no condition here tested how agents should act when a tool's effect is uncertain, but it is a reason to study that question. With semantically overlapping tools (Chattopadhyay et al., 2026; Rabinovich and Anaby-Tavor, 2025), single removals should start to matter. Interaction does not substitute for the missing information: declining to act fails because the evaluator requires the state change, probing fails at the step cap, and recovery is poor even in D2. Whether dry runs, more steps or structured diagnostics (Sigdel and Baral, 2026) would help is open.

For benchmark designers. The D2 effect flows through a dozen parameters in three backends and disappears when their constraints are written into the schema. A benchmark that encoded its enums and formats structurally, as τ^2 does, would isolate task understanding from prose parsing. Conversely, BFCL's multi-turn checker compares backend attributes with Python equality while its single-turn checker normalises case (Appendix A), so "`buy`" for "`Buy`" fails a task; 11 to 50% of the D2 losses begin that way (Section 6.2).

Information placement in LLM-facing interfaces. The results suggest reading a tool interface as a set of places where information can be stated, rather than as documentation to be written well. Names and descriptions served as identifiers of purpose, and any one of them sufficed. The schema served as the specification of constraints, and it did so at least as well as prose and for one model better: the same constraints written into the tool description instead (D9) brought two models back to baseline, left Gemma 4 15.5 points short and Qwen3 8 to 10, whereas the schema brought all four back. Redundancy across places is not waste: it let three carriers of purpose be removed one at a time without cost, and it protects against a model that reads one carrier less well than another. Admissible sets, formats and entity types belong in structural fields, where every model read them here; the prose remains for what structure cannot express. Some constraints cannot be stated in JSON Schema at all: an identifier that must exist in a database, a date that must follow another, a value that must match an earlier result. Those are stated by the tool's own outputs and errors, and the poor recovery rates of Section 6.4 say that this carrier is read badly, which makes informative errors and dry-run calls worth studying. The same framing separates two training targets, selection from purpose

information and construction from constraint information, which the behavioural split of Section 6.1 measures apart. What this paper tests is one benchmark's constraints moved into one place. The general question, what belongs in names, descriptions, schemas, outputs and context, is open.

8. Limitations

One run per condition. Two identical runs disagree on tens of tasks (Section 5.1), so this design cannot resolve differences of a few points. The D2, D5 and D6-versus-D2 effects are an order of magnitude larger than that. Every default configuration has a repeated D0 run on BFCL (Appendix B.4), but no ablation condition does. The conditions after D5 were not run in the same batch as the D0 and D2 they are compared with; the second D0 runs made alongside them, and the second Luna D2 run made alongside D8, are the checks on that (Appendix B). ToolSandbox was run once, and τ^2 once for D1, D3, D4 and D5, each with a simulated user whose variation enters every difference. τ^2's 50 tasks bound a single-trial null at 10 to 22 pp and the four-trial D2 null at 8.0 to 10.5 pp, which is coarser than the BFCL noise floor.

Two placements were tested, not every placement. D6 moves constraints from a parameter description into `enum`, `pattern` and a parameter name, and D9 moves them into the tool description; D6a and D6b separate D6's two moves (Appendix B.2). A constraint stated in the system prompt or in a tool's output was not tested, and the two placements tried did not serve every model equally, so "any carrier" holds only in the weak form that a stated constraint recovered most of the loss everywhere. The redundancy between parameter names and parameter descriptions was tested directly only by D8, and every condition after D5 was run once per model in its default setting.

D5 keeps the structure; the metrics are strict; the audit is a judgement. D5's collapse is partly a property of evaluators that cap steps and score final state. The audit was written after all three benchmarks had been run and their results seen, so it classifies after the fact and predicted nothing. Its rule is fixed in code and makes a testable prediction for any new task set, but BFCL's other multi-turn categories derive from the same 200 tasks and tools, so a predictive replication needs a different benchmark. Its "states a constraint" is a per-parameter reading of the schema text, written out so it can be disputed, and "copied" is verbatim occurrence, which misses a value the user stated in another form.

Provider and reasoning settings were not fully controlled. Routing was left to OpenRouter, a third of Qwen3's BFCL trajectories touched a second provider (the D2 effect is present on both subsets), and the reasoning switch sets no effort level.

9. Conclusion

Much of the information in tool documentation is stated more than once across the interface. What a tool does is stated in its name, in its description and usually in its signature, and on the three benchmarks tested that information survived the loss of any one of these, with two model-specific exceptions (Sections 5.1 and 5.3), but never the loss of all of them. Constraints on argument values are different. On BFCL, an allowed-value list, a format or an entity type was often stated in one parameter description and nowhere else, and removing that description removed the information, for the same six parameters in every model. Writing the same constraints into the schema restored baseline success with every parameter description blank; stating them in the tool description instead restored it for two of the four models, so the place still mattered, clearly for one model and possibly for two, and the schema was the place every model read. On two benchmarks whose constraints were already structural, or whose values the conversation supplied, the same removal had no detectable cost. It is the information that the models need, and the place it is stated is not always free to choose. Tool designers should state each constraint in a structural field as well as in words, and benchmark designers should know that a score can rest on where a convention happens to be written.

Reproducibility statement

The harness, the condition definitions (including the exact D6 transformation), the audit and analysis code and the paper sources are at `https://github.com/Eazydraft/exp-tool-semantics-ablation`, with BFCL,

ToolSandbox and τ^2-bench as pinned submodules. Every number, table and figure is computed from the 15 784 trajectory files under `runs/main/trajectories/`, each holding the exact tool schemas sent, every raw request and response, the benchmark's state and the verdict; `./run.sh browse` shows any trajectory with the schemas as the model saw them.

Acknowledgements

This study used the BFCL harness, backends and evaluator (Gorilla LLM contributors, 2026; Patil et al., 2025), ToolSandbox (Lu et al., 2024) and τ^2-bench unchanged; we thank their authors. Models were accessed through OpenRouter (OpenRouter, 2026). We thank Siddharth (`siddharth@lucioai.com`) for help with the code.

References

Anthropic. Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol, November 2024.

Anthropic. Introducing Claude Haiku 5.5. https://www.anthropic.com/claude-haiku-5-5, October 2026. Announced 2026-10-07. System card: https://www.anthropic.com/claude-haiku-5-5-system-card.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment, 2025. URL https://arxiv.org/abs/2506.07982.

Aniruddha Chattopadhyay, Kanishka Halder, and Partha Sarathi Roy. Robust function calling for LLM agents under interface drift, 2026. URL https://arxiv.org/abs/2610.23342.

Hugues Clouâtre. MCP parameter-description ablation: Token savings without accuracy loss. Zenodo, version 1.0.0, September 2026. Pre-registered experiment; code at https://github.com/clouatre-labs/param-description-experiments.

Olive Jean Dunn. Multiple comparisons among means. *Journal of the American Statistical Association*, 56 (293):52–64, 1961. doi: 10.1080/01621459.1961.10482090.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.

Google DeepMind. Gemma 4: Byte for byte, the most capable open models. https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/, April 2026. Announced 2026-04-02. Model card: https://huggingface.co/google/gemma-4-26b-a4b-it.

Gorilla LLM contributors. Gorilla: Berkeley function calling leaderboard source code. GitHub repository https://github.com/ShishirPatil/gorilla, 2026. Commit `6ea57973`, 2026-03-23, directory `berkeley-function-call-leaderboard`.

Ruocheng Guo, Kaiwen Dong, Xiang Gao, and Kamalika Das. Learning to rewrite tool descriptions for reliable LLM-agent tool use, 2026. URL https://arxiv.org/abs/2602.20426.

Cheng-Yu Hsieh, Si-An Chen, Chun-Liang Li, Yasuhisa Fujii, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. Tool documentation enables zero-shot tool-usage with large language models, 2023. URL https://arxiv.org/abs/2308.00675.

Reid T. Johnson, Michelle D. Pain, and Jordan D. West. Natural language tools: A natural language approach to tool calling in large language agents, 2025. URL https://arxiv.org/abs/2510.14453.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3102–3116. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.187.

Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities, 2024. URL https://arxiv.org/abs/2408.04682.

Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Jason Huang, Vishnu Suresh, Yixin Huang, Xiaowen Yu, Joseph E. Gonzalez, and Shishir G. Patil. BFCL V3: Multi-turn & multi-step function calling. Gorilla blog, UC Berkeley, September 2024. URL https://gorilla.cs.berkeley.edu/blogs/13_bfcl_v3_multi_turn.html. Released 2024-09-19, last updated 2024-12-10.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

OpenAI. Function calling and other API updates. https://openai.com/index/function-calling-and-other-api-updates/, June 2023.

OpenAI. Introducing GPT-6 Sol and Luna. https://openai.com/index/introducing-gpt-6-sol-and-luna/, September 2026.

OpenRouter. OpenRouter: A unified interface for LLMs. https://openrouter.ai, 2026. Accessed 2026-11-02.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs, 2023. URL https://arxiv.org/abs/2305.15334.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025. URL https://proceedings.mlr.press/v267/patil25a.html.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.16789.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. From exploration to mastery: Enabling LLMs to master tools via self-driven interactions. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025a. URL https://arxiv.org/abs/2410.08197. arXiv:2410.08197.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Tool learning with large language models: A survey. *Frontiers of Computer Science*, 2025b. doi: 10.1007/s11704-024-40678-2. URL https://arxiv.org/abs/2405.17935. arXiv:2405.17935.

Ella Rabinovich and Ateret Anaby-Tavor. On the robustness of agentic function calling. In *Proceedings of the 5th Workshop on Trustworthy NLP (TrustNLP 2025)*, pages 298–304. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.trustnlp-main.20. arXiv:2504.00914.

Jaideep Ray. The constraint tax: Measuring validity-correctness tradeoffs in structured outputs for small language models, 2026. URL https://arxiv.org/abs/2605.26128.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. arXiv:2302.04761.

Akshey Sigdel and Rista Baral. Schema first tool APIs for LLM agents: A controlled study of tool misuse, recovery, and budgeted performance, 2026. URL https://arxiv.org/abs/2603.13404.

Austin Wright, Henry Andrews, Ben Hutton, and Greg Dennis. JSON Schema: A media type for describing JSON documents, draft 2020-12. Internet-Draft, https://json-schema.org/draft/2020-12/json-schema-core, 2022.

Yuanjian Xu, Jinyi Han, Jianing Hao, Yuxuan Sun, Fanyang Lu, Richeng Xuan, Guang Zhang, and Zhichao Hu. AgentPerturb: Evaluating LLM agent robustness under controlled interaction perturbations, 2026. URL https://arxiv.org/abs/2610.21624.

Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Ion Stoica, Joseph E. Gonzalez, Tianjun Zhang, and Shishir G. Patil. Berkeley function calling leaderboard. Gorilla blog, UC Berkeley, 2024. URL https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html. Last updated 2024-08-19.

An Yang et al. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388. Model card for the July 2025 instruct update: https://huggingface.co/Qwen/Qwen3-30B-A3B-Instruct-2507.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2406.12045. arXiv:2406.12045.

Siyu Yuan, Kaitao Song, Jiangjie Chen, Xu Tan, Yongliang Shen, Kan Ren, Dongsheng Li, and Deqing Yang. EasyTool: Enhancing LLM-based agents with concise tool instruction. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 951–972. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.naacl-long.44. arXiv:2401.06201.

YiShan Zheng, Yuan Wu, and Yi Chang. ToolRobustBench: Stage-wise perturbation evaluation and failure diagnosis for tool-calling agents, 2026. URL https://arxiv.org/abs/2608.23635.

A. Additional results on BFCL

Memorisation check. Across the 1 515 to 3 927 tool calls each default configuration made in D3 and D5, and the 3 886 to 7 055 calls each reasoning variant made in D3, D4 and D5, no configuration emitted a single original tool name, and across D4 and D5 none emitted a single original parameter name. The only non-generic strings were eight malformed generic names from Qwen3 (`tool_6` for `tool_06`) and ten non-generic argument keys from Luna with reasoning. The models did not recall the hidden names from training.

Checker strictness. At the pinned commit, `standardize_string` in `bfcl_eval/eval_checker/ast_eval/ast_checker.py` removes spaces and punctuation and lower-cases both sides before the single-turn comparison. The multi-turn checker is stricter: `_compare_instances` in `bfcl_eval/eval_checker/multi_turn_eval/multi_turn_checker.py` tests every public attribute of the model's backend instance against the ground-truth instance with Python equality, and the multi-turn response checker matches serialised outputs without normalisation.

Haiku's D4 gain. Haiku with reasoning passes 29 tasks only in D4 against 7 only in D0 ($p = 0.0003$). The gain is absent with reasoning off (-0.5 pp) and shrinks to $+5.0$ pp ($p = 0.13$) against the repeated D0 run. It is also diffuse: the 29 tasks span 16 backend combinations, 19 involve the six parameters of Table 10 and 10 do not, and their D0 failures begin with a wrong argument in 12 cases, a never-called tool in 7 and no identifiable divergence in 10. We offer no mechanism for it.

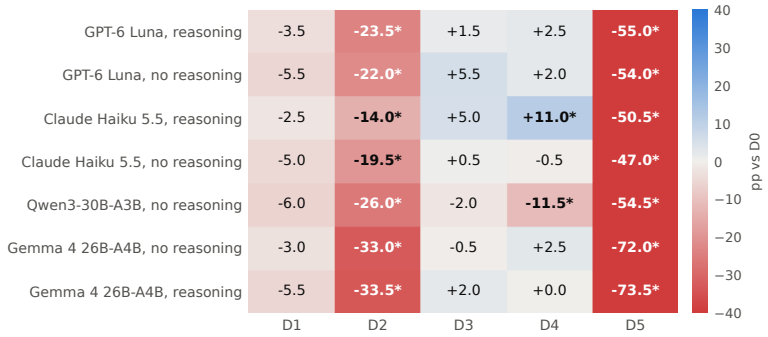


Figure 4 | Change in task success relative to D0, in percentage points. Starred, bold cells are significant at $p < 0.0014$ (exact McNemar test, Table 5).

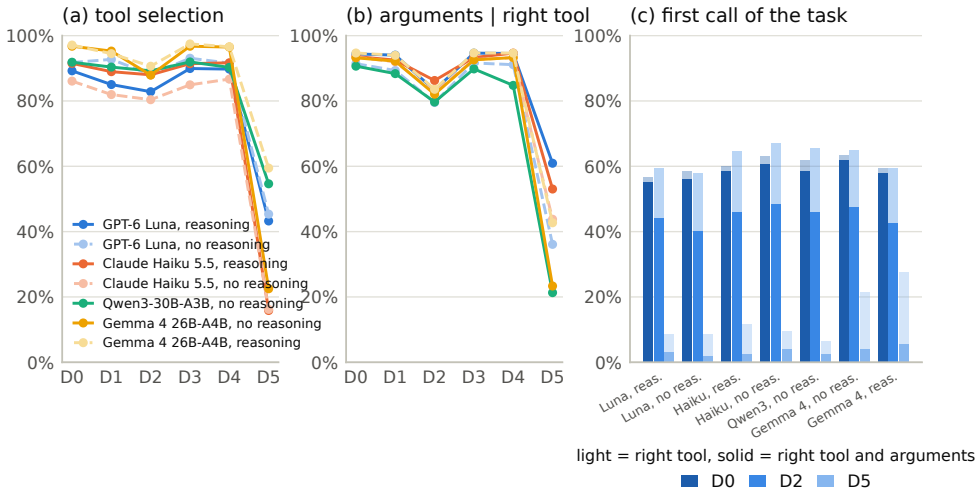


Figure 5 | (a) Tool-selection accuracy. (b) Argument accuracy given the right tool. (c) First call of the task: share of tasks whose first call used a ground-truth tool (light) and used it with the ground-truth arguments (solid), for D0, D2 and D5. Numbers in Table 8.

Table 5 | Each ablation versus D0 on the same 200 tasks: the two discordant cells, the paired difference Δ in percentage points with its 95% task-level bootstrap interval (2 000 resamples), the exact two-sided McNemar p, and the Bonferroni verdict for 35 comparisons ($p < 0.0014$). Each configuration's D1 to D5 share a launch batch with its D0.

Model	Cond.	both pass	D0 only	abl. only	both fail	Δ pp	95% CI	McNemar p	$p < 0.0014$
GPT-6 Luna, reasoning	D1	88	24	17	71	-3.5	[-9.5, 2.5]	0.3489	no
	D2	59	53	6	82	-23.5	[-30.0, -17.0]	<0.0001	**yes**
	D3	97	15	18	70	+1.5	[-4.0, 7.5]	0.7283	no
	D4	98	14	19	69	+2.5	[-3.0, 8.0]	0.4869	no
	D5	2	110	0	88	-55.0	[-61.5, -48.0]	<0.0001	**yes**
GPT-6 Luna, no reasoning	D1	77	31	20	72	-5.5	[-12.5, 1.5]	0.1608	no
	D2	48	60	16	76	-22.0	[-30.0, -14.0]	<0.0001	**yes**
	D3	85	23	34	58	+5.5	[-2.0, 13.0]	0.1849	no
	D4	86	22	26	66	+2.0	[-5.0, 9.0]	0.6655	no
	D5	0	108	0	92	-54.0	[-61.0, -47.0]	<0.0001	**yes**
Claude Haiku 5.5, reasoning	D1	79	22	17	82	-2.5	[-8.5, 3.5]	0.5224	no
	D2	59	42	14	85	-14.0	[-20.5, -7.0]	0.0002	**yes**
	D3	84	17	27	72	+5.0	[-1.0, 11.5]	0.1742	no
	D4	94	7	29	70	+11.0	[5.5, 17.0]	0.0003	**yes**
	D5	0	101	0	99	-50.5	[-57.5, -44.0]	<0.0001	**yes**
Claude Haiku 5.5, no reasoning	D1	69	25	15	91	-5.0	[-11.0, 1.5]	0.1539	no
	D2	48	46	7	99	-19.5	[-26.0, -12.5]	<0.0001	**yes**
	D3	76	18	19	87	+0.5	[-5.5, 6.5]	1.0000	no
	D4	74	20	19	87	-0.5	[-6.5, 5.5]	1.0000	no
	D5	0	94	0	106	-47.0	[-54.5, -40.0]	<0.0001	**yes**
Qwen3-30B-A3B, no reasoning	D1	78	31	19	72	-6.0	[-13.0, 0.5]	0.1189	no
	D2	46	63	11	80	-26.0	[-34.0, -18.5]	<0.0001	**yes**
	D3	85	24	20	71	-2.0	[-8.5, 4.5]	0.6516	no
	D4	74	35	12	79	-11.5	[-18.5, -5.0]	0.0011	**yes**
	D5	0	109	0	91	-54.5	[-61.5, -48.0]	<0.0001	**yes**
Gemma 4 26B-A4B, no reasoning	D1	130	14	8	48	-3.0	[-7.5, 1.5]	0.2863	no
	D2	73	71	5	51	-33.0	[-40.5, -26.0]	<0.0001	**yes**
	D3	136	8	7	49	-0.5	[-4.5, 3.5]	1.0000	no
	D4	141	3	8	48	+2.5	[-0.5, 5.5]	0.2266	no
	D5	0	144	0	56	-72.0	[-78.5, -66.0]	<0.0001	**yes**
Gemma 4 26B-A4B, reasoning	D1	133	19	8	40	-5.5	[-10.5, -0.5]	0.0522	no
	D2	77	75	8	40	-33.5	[-41.5, -26.0]	<0.0001	**yes**
	D3	146	6	10	38	+2.0	[-1.5, 6.0]	0.4545	no
	D4	148	4	4	44	+0.0	[-2.5, 2.5]	1.0000	no
	D5	5	147	0	48	-73.5	[-79.5, -67.0]	<0.0001	**yes**

Table 6 | Reasoning tokens per task by condition, mean over the 200 tasks.

Model	D0	D1	D2	D3	D4	D5
GPT-6 Luna, reasoning	330	368	390	328	335	2,294
GPT-6 Luna, no reasoning	0	0	0	0	0	0
Claude Haiku 5.5, reasoning	517	548	559	550	550	1,258
Claude Haiku 5.5, no reasoning	0	0	0	0	0	0
Qwen3-30B-A3B, no reasoning	0	0	0	0	0	0
Gemma 4 26B-A4B, no reasoning	0	0	0	0	2	0
Gemma 4 26B-A4B, reasoning	1,110	1,521	2,554	1,328	1,316	8,659

Table 7 | Documentation sensitivity score $S_m = \frac{1}{N}\sum_i (P_{m,0} - P_{m,i})/P_{m,0}$, the mean relative drop from D0 over D1–D4 and over D1–D5. Because it averages a large D2 loss with near-zero changes elsewhere, it understates the D2 effect and is reported for completeness only.

Model	S (D1–D4)	S (D1–D5)
GPT-6 Luna, reasoning	0.103	0.279
GPT-6 Luna, no reasoning	0.093	0.274
Claude Haiku 5.5, reasoning	0.002	0.202
Claude Haiku 5.5, no reasoning	0.130	0.304
Qwen3-30B-A3B, no reasoning	0.209	0.367
Gemma 4 26B-A4B, no reasoning	0.118	0.294
Gemma 4 26B-A4B, reasoning	0.122	0.291

Table 8 | Tool selection versus argument construction (%). "Sel." is tool-selection accuracy, "Arg." argument accuracy given the right tool, "1st tool" the share of tasks whose first call used a ground-truth tool, "1st exact" the share whose first call matched a ground-truth call exactly.

	D0				D2				D5			
Model	Sel.	Arg.	1st tool	1st exact	Sel.	Arg.	1st tool	1st exact	Sel.	Arg.	1st tool	1st exact
GPT-6 Luna, reasoning	89.2	94.4	56.5	55.0	82.8	83.3	59.5	44.0	43.3	60.9	8.5	3.0
GPT-6 Luna, no reasoning	91.8	91.4	58.5	56.0	88.1	79.5	58.0	40.0	45.4	36.1	8.5	2.0
Claude Haiku 5.5, reasoning	91.5	93.6	60.0	58.5	88.0	86.3	64.5	46.0	15.8	53.0	11.5	2.5
Claude Haiku 5.5, no reasoning	86.1	93.4	63.0	60.5	80.4	82.6	67.0	48.5	16.2	43.8	9.5	4.0
Qwen3-30B-A3B, no reasoning	91.9	90.7	62.0	58.5	89.2	79.7	65.5	46.0	54.6	21.3	6.5	2.5
Gemma 4 26B-A4B, no reasoning	96.8	93.2	63.5	62.0	87.9	82.1	65.0	47.5	22.5	23.3	21.5	4.0
Gemma 4 26B-A4B, reasoning	97.1	94.7	59.5	58.0	90.6	83.6	59.5	42.5	59.5	42.7	27.5	5.5

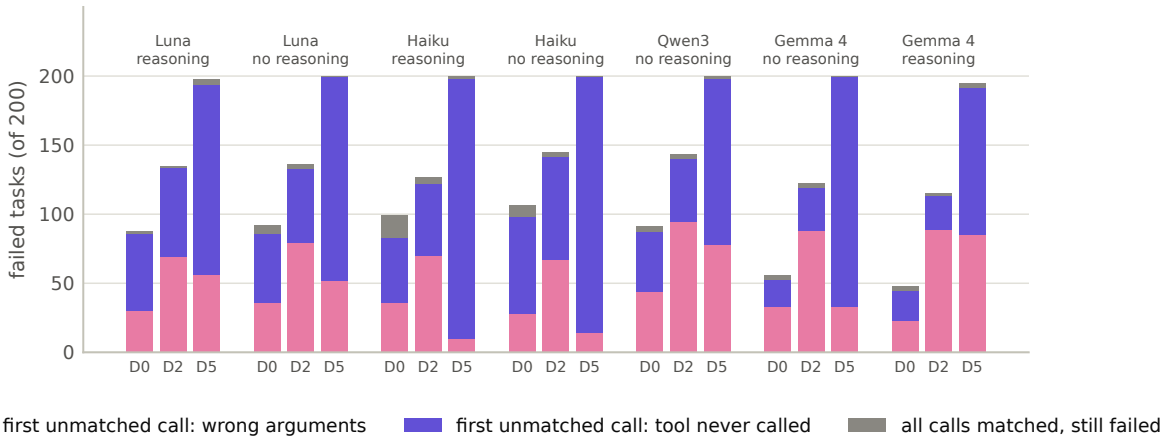


Figure 6 | Earliest divergence in failed tasks: the first ground-truth call without an exact match, classified by whether the tool was called with wrong arguments (construction failure) or never called (selection failure); "all calls matched" means every ground-truth call was matched but the task still failed, usually through extra state-changing calls or a call made in a later turn than required.

Table 9 | Earliest divergence among failed BFCL tasks (counts). "W.A.": wrong arguments; "N.C.": tool never called; "None": every ground-truth call matched but the task failed.

	D0			D2			D5		
Model	W.A.	N.C.	None	W.A.	N.C.	None	W.A.	N.C.	None
GPT-6 Luna, reasoning	30	56	2	69	65	1	56	138	4
GPT-6 Luna, no reasoning	36	50	6	79	54	3	52	148	0
Claude Haiku 5.5, reasoning	36	47	16	70	52	5	10	188	2
Claude Haiku 5.5, no reasoning	28	70	8	67	75	3	14	186	0
Qwen3-30B-A3B, no reasoning	44	43	4	95	45	3	78	120	2
Gemma 4 26B-A4B, no reasoning	33	20	3	88	31	3	33	167	0
Gemma 4 26B-A4B, reasoning	23	22	3	89	24	2	85	107	3

Table 10 | Parameters whose values go wrong when their description is removed. For every ground-truth call matched by the right tool but wrong arguments, the differing parameters were recorded. Counts are pooled over the seven configurations (1 400 task runs per condition), sorted by D2−D0; D2 counts per configuration on the right (−: reasoning off, +: on). The second column quotes the description D2 removed.

Tool.parameter	Removed description	pooled, 7 configs		D2 count per configuration						
		D2	D0	Luna	Luna$^-$	Haiku	Haiku$^-$	Qwen3	Gemma	Gemma$^+$
`place_order.order_type`	*Type of the order (Buy/Sell).*	**186**	0	26	28	27	22	27	28	28
`startEngine.ignitionMode`	*The ignition mode of the vehicle. [Enum]: ["START", "STOP"]*	**174**	0	25	22	14	26	28	30	29
`estimate_distance.cityA`	*The zipcode of the first city.*	**134**	7	20	20	20	20	16	19	19
`estimate_distance.cityB`	*The zipcode of the second city.*	**134**	7	20	20	20	20	16	19	19
`post_tweet.tags`	*List of tags for the tweet. Tag name should start with #. This is only relevant if the user wants to add tags to the tweet.*	**130**	4	22	24	18	15	12	20	19
`post_tweet.mentions`	*List of users mentioned in the tweet. Mention name should start with @. This is only relevant if the user wants to add mentions to the tweet.*	**75**	5	15	14	11	6	8	11	10
`get_available_stocks.sector`	*The sector to retrieve stocks from (e.g., 'Technology').*	**17**	0	3	4	0	2	4	1	3
`post_tweet.content`	*Content of the tweet.*	**62**	47	6	5	17	17	3	11	3
`lockDoors.door`	*The list of doors to lock or unlock. [Enum]: ["driver", "passenger", "rear_left", "rear_right"]*	**22**	7	2	7	1	1	4	4	3
`get_zipcode_based_on_city.city`	*The name of the city.*	**14**	0	1	1	4	1	4	1	2

Table 11 | Success (%) on the 91 tasks whose ground truth uses at least one of the six parameters of Table 10 ("six") and on the other 109 ("other"). The first column counts the tasks that pass in D0 and fail in D2 and, of those, how many are six-parameter tasks; the last column counts how many of those D2 losses begin with a value that differs from the ground truth only in case. D6 to D8 exist for each model's default reasoning setting.

	D0-only tasks	D0		D2		D6		D7		D8		case-only
Model	six / all	six	other	six	other	six	other	six	other	six	other	D2 losses
GPT-6 Luna, reasoning	44/53	51.6	59.6	4.4	56.0	50.5	56.9	49.5	54.1	3.3	40.4	16/53
GPT-6 Luna, no reasoning	50/60	59.3	49.5	6.6	53.2	–	–	–	–	–	–	18/60
Claude Haiku 5.5, reasoning	37/42	46.2	54.1	9.9	58.7	49.5	57.8	51.6	57.8	3.3	44.0	21/42
Claude Haiku 5.5, no reasoning	29/46	34.1	57.8	3.3	47.7	–	–	–	–	–	–	15/46
Qwen3-30B-A3B, no reasoning	50/63	60.4	49.5	8.8	45.0	59.3	44.0	49.5	40.4	2.2	29.4	12/63
Gemma 4 26B-A4B, no reasoning	64/71	75.8	68.8	5.5	67.0	75.8	65.1	64.8	64.2	0.0	31.2	8/71
Gemma 4 26B-A4B, reasoning	68/75	81.3	71.6	7.7	71.6	–	–	–	–	–	–	28/75

B. Conditions after D5: structure, placement and repeated runs

The transformations. Every condition after D5 blanks every parameter description, as D2 does, in order to test the explanation that Section 6.2 arrives at. D6 then writes every prose-only constraint of the BFCL schemas into a structural field. Eleven parameters whose description holds an allowed-value list receive an `enum` (`activateParkingBrake.mode`, `adjustClimateControl.mode` and `unit`, `compute_exchange_rate.base_currency` and `target_currency`, `displayCarStatus.option`, `get_nearest_airport_by_city.location`, `lockDoors.door`, `place_order.order_type`, `setHeadlights.mode`, `startEngine.ignitionMode`), `post_tweet.tags` and `mentions` receive the item patterns `^#` and `^@`, and `estimate_distance.cityA` and `cityB` are renamed `zipcodeA` and `zipcodeB`: 15 parameters, 13 with a new structural field and two with a new name. D6a and D6b split D6 into its two moves: D6a adds the `enum`s and patterns but keeps the names `cityA` and `cityB`, and D6b only renames them. D9 keeps the schema as in D2 and instead appends each of the same constraints, as a sentence, to the tool's description ("The parameter order_type must be one of: Buy, Sell."). The information that D6 moves into the schema is thus moved into another piece of prose. D7 keeps only the six descriptions of Table 10 and blanks the rest. D8 removes parameter names and parameter descriptions together while keeping tool names and descriptions; if the names stated nothing that the descriptions did not, D8 should perform like D2.

B.1. D6 and D7: constraints written into the schema

Writing the constraints into the schema restores baseline with every description blank. If the descriptions that matter are the sole statements of constraints the schema could express, two things should follow: D6 (descriptions blank, constraints written into the schema) should perform like D0, and D7 (descriptions blank except the six) should recover most but not all of the loss. Both hold (Figure 7, Table 12). D6 lands within 3.5 points of D0 for every model; no D6–D0 difference is significant, and every D6–D2 difference is, recovering 17.5 to 31.0 points. D7 recovers most of the D2 loss but stays 4 to 10 points below D0 for three models. What still goes wrong under D7 is the remaining prose-only enums that only D6 converts, led by `lockDoors.door`. The models read the constraint rather than having it enforced on them. Counting every value emitted for the 13 constrained parameters, the share outside the constraint is 37 to 69% in D2 and at most 0.7% in D6 for every model, and three models were already at 0.0% in D0, where the constraint was prose that no decoder could enforce. Enforcement is therefore not needed to explain the recovery, which matters because enforcement is not free (Ray, 2026).

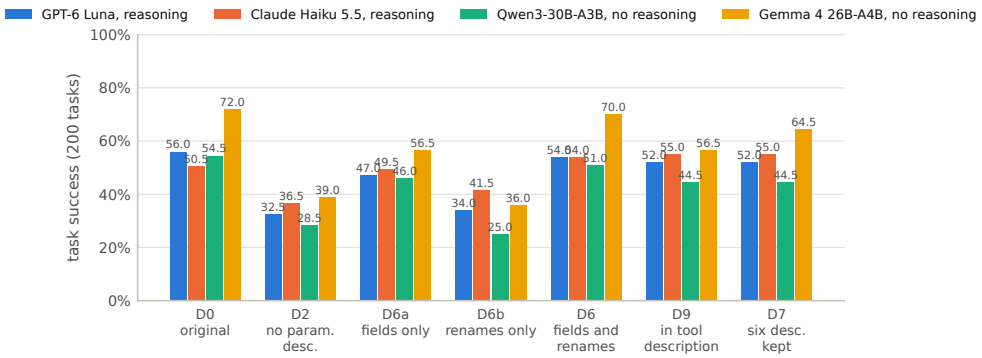


Figure 7 | The conditions after D5, each model in its default reasoning setting. All blank every parameter description, as D2 does. D6 moves the prose-only constraints into the schema; D6a adds only its `enum`s and patterns and D6b only its two renames; D9 states the same constraints in the tool description instead; D7 keeps only the six descriptions of Table 10. Paired tests in Table 12.

B.2. D6a, D6b and D9: which move, and does the place matter

The structural fields carry most of the recovery; the renames matter only alongside them. D6a keeps D6's `enum`s and patterns and leaves `cityA` and `cityB` as they were. It recovers +13.0 to +17.5 pp over D2 for the four models (every $p < 0.0001$; Table 12), which is most of D6's recovery, but it stops −13.5 to −4.5 pp short of D6 and −15.5 to −1.0 pp short of D0. D6b, the two renames alone, recovers nothing that the noise floor cannot explain: −3.5 to +5.0 pp against D2 (every $p \geq 0.09$). Against D0, D6a's shortfall is significant at the corrected threshold only for Gemma 4 (−15.5 pp, $p < 0.0001$; −16.5 pp against the second

Table 12 | Each condition after D5 against D0 and against D2 on the same 200 tasks (each model in its default reasoning setting): success of the base condition and of the follow-up condition, the discordant cells, the paired difference with its 95% bootstrap interval, and the exact McNemar p. None of these conditions was run in the same batch as D0 to D5; Table 13 compares them with a second D0 run made alongside them. The p-values are uncorrected; the Bonferroni threshold for the 52 comparisons after D5 is $p < 0.001$.

Model	Comparison	base %	cond. %	base only	cond. only	Δ pp	95% CI	p
GPT-6 Luna, reasoning	D6 vs D0	56.0	54.0	19	15	-2.0	[-7.5, 4.0]	0.6076
	D6 vs D2	32.5	54.0	6	49	+21.5	[15.0, 28.0]	<0.0001
	D6a vs D0	56.0	47.0	28	10	-9.0	[-15.0, -3.0]	0.0051
	D6a vs D2	32.5	47.0	12	41	+14.5	[7.5, 21.5]	<0.0001
	D6b vs D0	56.0	34.0	50	6	-22.0	[-29.0, -15.5]	<0.0001
	D6b vs D2	32.5	34.0	7	10	+1.5	[-2.5, 5.5]	0.6291
	D9 vs D0	56.0	52.0	21	13	-4.0	[-10.0, 2.0]	0.2295
	D9 vs D2	32.5	52.0	8	47	+19.5	[13.0, 26.5]	<0.0001
	D7 vs D0	56.0	52.0	23	15	-4.0	[-10.0, 2.0]	0.2559
	D7 vs D2	32.5	52.0	7	46	+19.5	[13.0, 26.0]	<0.0001
Claude Haiku 5.5, reasoning	D6 vs D0	50.5	54.0	14	21	+3.5	[-2.0, 9.5]	0.3105
	D6 vs D2	36.5	54.0	7	42	+17.5	[11.0, 24.0]	<0.0001
	D6a vs D0	50.5	49.5	20	18	-1.0	[-7.0, 5.0]	0.8714
	D6a vs D2	36.5	49.5	8	34	+13.0	[7.0, 19.0]	<0.0001
	D6b vs D0	50.5	41.5	39	21	-9.0	[-16.0, -1.5]	0.0273
	D6b vs D2	36.5	41.5	9	19	+5.0	[0.0, 10.0]	0.0872
	D9 vs D0	50.5	55.0	13	22	+4.5	[-1.0, 10.5]	0.1755
	D9 vs D2	36.5	55.0	10	47	+18.5	[11.5, 25.0]	<0.0001
	D7 vs D0	50.5	55.0	16	25	+4.5	[-2.0, 11.0]	0.2110
	D7 vs D2	36.5	55.0	13	50	+18.5	[11.0, 25.5]	<0.0001
Qwen3-30B-A3B, no reasoning	D6 vs D0	54.5	51.0	23	16	-3.5	[-10.0, 2.5]	0.3368
	D6 vs D2	28.5	51.0	6	51	+22.5	[16.0, 29.5]	<0.0001
	D6a vs D0	54.5	46.0	35	18	-8.5	[-16.0, -2.0]	0.0270
	D6a vs D2	28.5	46.0	7	42	+17.5	[11.0, 24.0]	<0.0001
	D6b vs D0	54.5	25.0	70	11	-29.5	[-37.5, -22.0]	<0.0001
	D6b vs D2	28.5	25.0	16	9	-3.5	[-8.5, 1.5]	0.2295
	D9 vs D0	54.5	44.5	31	11	-10.0	[-17.0, -4.0]	0.0029
	D9 vs D2	28.5	44.5	7	39	+16.0	[9.5, 22.5]	<0.0001
	D7 vs D0	54.5	44.5	32	12	-10.0	[-16.5, -3.5]	0.0037
	D7 vs D2	28.5	44.5	14	46	+16.0	[9.0, 23.5]	<0.0001
Gemma 4 26B-A4B, no reasoning	D6 vs D0	72.0	70.0	13	9	-2.0	[-6.5, 2.5]	0.5235
	D6 vs D2	39.0	70.0	9	71	+31.0	[23.5, 39.0]	<0.0001
	D6a vs D0	72.0	56.5	37	6	-15.5	[-21.5, -9.5]	<0.0001
	D6a vs D2	39.0	56.5	16	51	+17.5	[10.0, 25.5]	<0.0001
	D6b vs D0	72.0	36.0	76	4	-36.0	[-43.0, -29.0]	<0.0001
	D6b vs D2	39.0	36.0	13	7	-3.0	[-7.5, 1.5]	0.2632
	D9 vs D0	72.0	56.5	36	5	-15.5	[-21.5, -9.5]	<0.0001
	D9 vs D2	39.0	56.5	13	48	+17.5	[10.5, 25.0]	<0.0001
	D7 vs D0	72.0	64.5	25	10	-7.5	[-13.0, -2.0]	0.0167
	D7 vs D2	39.0	64.5	7	58	+25.5	[18.5, 32.5]	<0.0001

D0 run); Luna's and Qwen3's ($p = 0.0051$ and $p = 0.027$) are not. The two moves are not additive because the tasks that need them overlap. Twenty tasks call `estimate_distance`, and 19 of those also use one of the other five parameters, so a task that gets the zip codes right still fails on an enum, and a task that gets the enums right still fails on the city names. Under D6a the models pass 0, 0, 3 and 0 of the 20 zip-code tasks (Luna, Haiku, Qwen3, Gemma 4), under D6b 4, 10, 4 and 2, and under D6, with both moves, 9, 11, 11 and 17, which matches D0's 9, 13, 14 and 15. The zip-code tasks are where D6 pulls ahead of D6a, but not the only place. Per model, 27, 21, 21 and 35 tasks pass in D6 and fail in D6a, of which 9, 11, 9 and 17 are zip-code tasks; 13, 12, 11 and 8 tasks go the other way, almost none of them zip-code tasks. Outside the zip-code tasks the two conditions therefore differ by a net +2.5, −1.0, +1.0 and +5.0 pp in D6's favour, which is within each model's noise floor except perhaps Gemma 4's, and the two conditions were not run together, so part of that residual may be run-to-run variation. What the split supports is narrower than a clean attribution: the structural fields recover the parameters they cover, the renames recover the two parameters whose constraint was an entity type no field expresses, and the zip-code tasks need both.

Stating the constraints in the tool description works for two models and less well for two. D9 keeps the schema exactly as in D2 and appends every D6 constraint, including "cityA is a zip code", to the tool description as a sentence. It recovers +16.0 to +19.5 pp over D2 (every $p < 0.0001$), about as much as

D6a. Against D0 it is −15.5 to +4.5 pp. Luna and Haiku are back at baseline (−4.0 and +4.5 pp, $p \geq 0.18$). Gemma 4's −15.5 pp passes the corrected threshold of $p < 0.001$ for this family ($p < 0.0001$). Qwen3's −10.0 pp ($p = 0.0029$) does not, but its interval excludes zero, so it is short of baseline without being significantly so at the corrected level. Because D9 was not run in the same batch as the first D0, Table 13 repeats the comparison against the second D0 run made alongside D6a, D6b and D9: Gemma 4's shortfall holds (−16.5 pp, $p < 0.0001$), Qwen3's is −8.0 pp with interval [−14.0, −2.5] ($p = 0.0113$), again excluding zero, and Luna's is −4.5 pp ($p = 0.1877$). Gemma 4 passes 10 of the 20 zip-code tasks under D9 against 17 under D6, so the sentence about zip codes in the tool description served it less well than a parameter name saying the same. The placement of a constraint therefore mattered clearly for one of the four models and possibly for a second, and for all four a constraint stated anywhere the model could read recovered most of what blanking the descriptions had cost. "Any carrier" is supported in that weaker form: the information must be stated, and the schema was the place every model read best, but prose in a neighbouring field was far better than nothing.

Table 13 | D6a, D6 and D9 against the second D0 run that was made alongside them, for the three models with such a run (Haiku's second D0 predates these conditions). Same layout as Table 12, with the equivalence bound; p uncorrected.

Model	Cond.	second D0 %	cond. %	D0 only	cond. only	Δ pp	95% CI	bound	p
GPT-6 Luna	D6a	56.5	47.0	30	11	-9.5	[-15.5, -3.0]	15.5	0.0043
	D6	56.5	54.0	19	14	-2.5	[-8.0, 3.0]	8.0	0.4869
	D9	56.5	52.0	23	14	-4.5	[-10.5, 1.5]	10.5	0.1877
Qwen3-30B-A3B	D6a	52.5	46.0	31	18	-6.5	[-13.5, 0.0]	13.5	0.0854
	D6	52.5	51.0	23	20	-1.5	[-8.5, 4.5]	8.5	0.7608
	D9	52.5	44.5	26	10	-8.0	[-14.0, -2.5]	14.0	0.0113
Gemma 4 26B-A4B	D6a	73.0	56.5	39	6	-16.5	[-22.5, -10.5]	22.5	<0.0001
	D6	73.0	70.0	17	11	-3.0	[-8.0, 2.0]	8.0	0.3449
	D9	73.0	56.5	38	5	-16.5	[-22.5, -10.5]	22.5	<0.0001

B.3. D8: parameter names and descriptions removed together

Removing parameter names and descriptions together costs more than either. D8 blanks every parameter description and makes every parameter name generic while keeping tool names and descriptions, so it removes the two parameter-level carriers that D2 and D4 removed one at a time. Success falls to 17.0 to 25.5%, which is 9.0 to 22.0 pp below D2 (every $p \leq 0.0005$; Table 14) and 26.0 to 57.5 pp below D4. The extra loss is not the six parameters' doing. On the 109 tasks that use none of them, where D2 moved success by at most 4.6 pp, D8 costs 10.1 to 37.6 pp (Table 11). The parameters that newly go wrong, `book_flight`'s arguments and `lockDoors.unlock`, are ones whose values the model can no longer assign to the right slot once both the name and the description are gone. What a parameter *is* was stated twice, by its name and by its description. Removing either alone cost nothing (D4, for three models) or cost only the constraints that the descriptions alone stated (D2); removing both leaves the type as the only statement. D8 was not run in the same batch as D2. As a check, a second Luna D2 run made alongside D8 differs from the first D2 run by −2.5 pp, and D8 is −6.5 pp below it ($p = 0.0192$).

B.4. Repeated baseline runs

Every default configuration has its own noise floor. Each of the four default configurations was run a second time at D0 on the same 200 tasks with the same settings (Table 15). The second run differs from the first by 0.5 to 6.0 pp in absolute value, with 8 to 34 discordant tasks, and no difference is significant after correction. Claude Haiku 5.5's two runs are the widest pair (9 tasks passing only in the first run and 21 only in the second; Table 16 compares every condition with both); the other three models sit closer. The floors are all of the same order as the D1 and D3 differences of Section 5.1 and an order of magnitude below the D2 and D5 effects, so the single-condition verdicts drawn against Haiku's floor hold against each model's own.

Table 14 | D8 (generic parameter names and blank parameter descriptions together, tool names and descriptions kept) against D0, D2 and D4 on the same 200 tasks (each model in its default reasoning setting): success of the base condition and of D8, the discordant cells, the paired difference with its 95% bootstrap interval, its equivalence bound and the exact McNemar p (uncorrected; the threshold for the 52 comparisons after D5 is $p < 0.001$). D8 was not run in the same batch as D0 to D5.

Model	Comparison	base %	D8 %	base only	D8 only	Δ pp	95% CI	bound	p
GPT-6 Luna, reasoning	D8 vs D0	56.0	23.5	71	6	-32.5	[-39.5, -25.0]	39.5	<0.0001
	D8 vs D2	32.5	23.5	22	4	-9.0	[-14.0, -4.0]	14.0	0.0005
	D8 vs D4	58.5	23.5	76	6	-35.0	[-42.5, -28.0]	42.5	<0.0001
Claude Haiku 5.5, reasoning	D8 vs D0	50.5	25.5	58	8	-25.0	[-32.0, -17.5]	32.0	<0.0001
	D8 vs D2	36.5	25.5	27	5	-11.0	[-16.5, -6.0]	16.5	0.0001
	D8 vs D4	61.5	25.5	76	4	-36.0	[-43.0, -28.5]	43.0	<0.0001
Qwen3-30B-A3B, no reasoning	D8 vs D0	54.5	17.0	82	7	-37.5	[-45.0, -30.0]	45.0	<0.0001
	D8 vs D2	28.5	17.0	31	8	-11.5	[-17.5, -5.5]	17.5	0.0003
	D8 vs D4	43.0	17.0	58	6	-26.0	[-33.5, -19.0]	33.5	<0.0001
Gemma 4 26B-A4B, no reasoning	D8 vs D0	72.0	17.0	114	4	-55.0	[-62.5, -47.5]	62.5	<0.0001
	D8 vs D2	39.0	17.0	47	3	-22.0	[-28.5, -16.0]	28.5	<0.0001
	D8 vs D4	74.5	17.0	118	3	-57.5	[-64.5, -50.5]	64.5	<0.0001

Table 15 | Run-to-run noise per model: a second D0 run against the first on the same 200 tasks in the default reasoning setting. Success of each run, the tasks passing in only one of them, the paired difference with its 95% bootstrap interval and the exact McNemar p. The second runs of Luna, Qwen3 and Gemma 4 were made together with D6a, D6b and D9.

Model	run 1 %	run 2 %	run 1 only	run 2 only	Δ pp	95% CI	p
GPT-6 Luna, reasoning	56.0	56.5	13	14	+0.5	[-4.5, 5.5]	1.0000
Claude Haiku 5.5, reasoning	50.5	56.5	9	21	+6.0	[1.0, 11.5]	0.0428
Qwen3-30B-A3B, no reasoning	54.5	52.5	19	15	-2.0	[-7.5, 4.0]	0.6076
Gemma 4 26B-A4B, no reasoning	72.0	73.0	3	5	+1.0	[-1.5, 4.0]	0.7266

Table 16 | Claude Haiku 5.5's repeated D0 run against the first, and each condition against both D0 runs. Same 200 tasks, same settings; no comparison with the second run shares a batch. Table 15 gives the repeated runs of the other models.

Comparison	both pass	first only	second only	both fail	Δ pp	95% CI	McNemar p
D0 run 1 vs D0 run 2	92	9	21	78	+6.0	[1.0, 11.5]	0.0428
D0 run 1 vs D1	79	22	17	82	-2.5	[-8.5, 3.5]	0.5224
D0 run 2 vs D1	86	27	10	77	-8.5	[-14.5, -3.0]	0.0076
D0 run 1 vs D2	59	42	14	85	-14.0	[-20.5, -7.0]	0.0002
D0 run 2 vs D2	61	52	12	75	-20.0	[-27.0, -13.0]	<0.0001
D0 run 1 vs D3	84	17	27	72	+5.0	[-1.0, 11.5]	0.1742
D0 run 2 vs D3	97	16	14	73	-1.0	[-6.0, 4.5]	0.8555
D0 run 1 vs D4	94	7	29	70	+11.0	[5.5, 17.0]	0.0003
D0 run 2 vs D4	100	13	23	64	+5.0	[-1.0, 11.0]	0.1325
D0 run 1 vs D5	0	101	0	99	-50.5	[-57.5, -44.0]	<0.0001
D0 run 2 vs D5	0	113	0	87	-56.5	[-63.0, -49.5]	<0.0001
D0 run 1 vs D6	87	14	21	78	+3.5	[-2.0, 9.5]	0.3105
D0 run 2 vs D6	95	18	13	74	-2.5	[-8.0, 3.0]	0.4731
D0 run 1 vs D7	85	16	25	74	+4.5	[-2.0, 11.0]	0.2110
D0 run 2 vs D7	96	17	14	73	-1.5	[-7.0, 3.5]	0.7201

C. The information audit in detail

Parameters whose description is the only statement of a constraint. The audit's per-parameter judgement is the table `CONSTRAINT_KIND` in `ablation/audit.py`; the parameters it lists that occur in the ground truth are, per benchmark:

BFCL multi-turn: `activateParkingBrake.mode`, `add_to_watchlist.stock`, `authenticate_travel.grant_type`, `book_flight.travel_date`, `cat.file_name`, `cd.folder`, `compute_exchange_rate.base_currency`, `compute_exchange_rate.target_currency`, `cp.destination`, `create_ticket.priority`, `displayCarStatus.option`, `estimate_distance.cityA`, `estimate_distance.cityB`, `estimate_drive_feasibility_by_mileage.distance`, `fillFuelTank.fuelAmount`, `get_flight_cost.travel_class`, `get_flight_cost.travel_date`, `get_flight_cost.travel_from`, `get_flight_cost.travel_to`, `get_nearest_airport_by_city.location`, `grep.file_name`, `lockDoors.door`, `mkdir.dir_name`, `mv.destination`, `mv.source`, `place_order.order_type`, `post_tweet.mentions`, `post_tweet.tags`, `pressBrakePedal.pedalPosition`, `register_credit_card.expiration_date`, `rmdir.dir_name`,

Table 17 | How many carriers state each tool's purpose. A tool instance is one tool in one task's ground truth; every tool has a name and a description, and its signature counts as a third carrier when no tool offered alongside it in that task has the same parameter names and types. The last column lists the tools that most often share a signature.

Benchmark	tool instances	3 carriers	2 carriers	shared	most often shared
BFCL multi-turn	1082	759	323	30%	`get_stock_info`, `get_order_details`, `check_tire_pressure`, `get_watchlist`
ToolSandbox	154	69	85	55%	`get_current_timestamp`, `search_contacts`, `modify_contact`, `set_cellular_service_status`
τ^2-bench airline	85	45	40	47%	`get_reservation_details`, `search_direct_flight`, `cancel_reservation`

`setCruiseControl.distanceToNextVehicle`, `setCruiseControl.speed`, `setHeadlights.mode`, `set_budget_limit.budget_limit`, `set_navigation.destination`, `startEngine.ignitionMode`, `tail.file_name`, `touch.file_name`, `verify_traveler_information.date_of_birth`, `wc.mode`.

ToolSandbox: `add_reminder.reminder_timestamp`, `modify_reminder.reminder_timestamp`.

τ^2-bench airline: `book_reservation.destination`, `book_reservation.origin`, `calculate.expression`, `search_direct_flight.date`, `search_direct_flight.destination`, `search_direct_flight.origin`.

On BFCL, a boolean is the only structural constraint any ground-truth parameter carries, so no BFCL argument has a constraint stated both in prose and in structure. "Copied" takes its outputs from the first configuration whose D0 run passed the task. For 71 BFCL ground-truth calls, 40 ToolSandbox pinned values and 26 τ^2 actions the producing call could not be located in such a trajectory, and the audit then counts every output the trajectory received.

D. Tool-use behaviour on BFCL

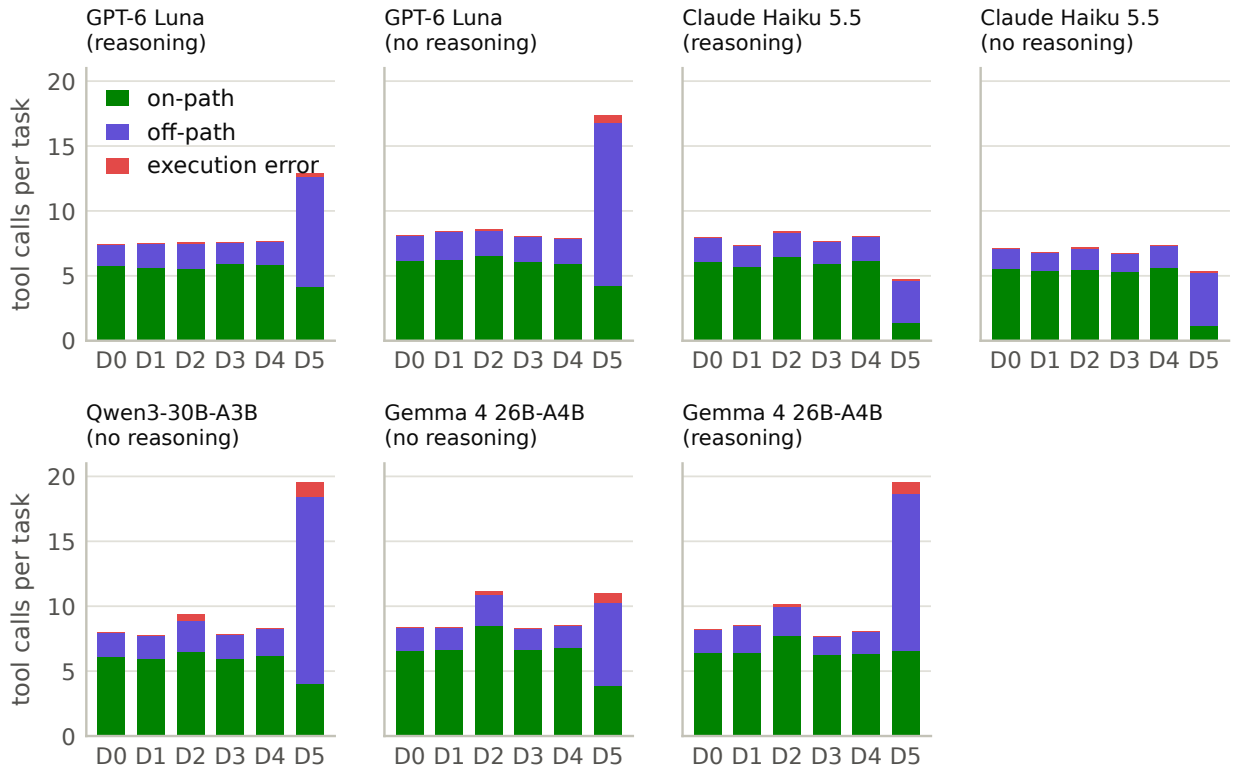


Figure 8 | Composition of tool calls per task (mean over all 200 tasks, failures included): on-path calls, off-path calls that executed without error, and calls that returned an execution error.

D5 behaviour in detail. The models split into two strategies. Haiku makes fewer calls than in D0 (4.7 against 7.9 with reasoning, 5.3 against 7.1 without), makes no call in the first turn of 34.5% of tasks (18.0%

Table 18 | Tool-use behaviour, mean per task over all 200 tasks (failures included). A repeated call has identical tool and arguments to an earlier call in the same task; "calls/pass" is the mean over passing tasks only.

Model	Cond.	calls	steps	on-path	off-path	failed	repeated	distinct tools	calls/pass
GPT-6 Luna, reasoning	D0	7.4	10.2	5.8	1.6	0.01	0.50	6.2	7.4
	D1	7.5	10.2	5.6	1.9	0.01	0.58	6.0	7.2
	D2	7.6	10.6	5.6	1.9	0.15	0.52	6.0	7.8
	D3	7.6	10.4	5.9	1.7	0.00	0.65	6.2	7.6
	D4	7.7	10.5	5.8	1.8	0.01	0.59	6.3	7.5
	D5	12.9	14.7	4.2	8.4	0.28	0.86	9.1	22.5
GPT-6 Luna, no reasoning	D0	8.1	11.5	6.2	1.9	0.03	0.86	6.4	7.9
	D1	8.4	11.8	6.2	2.1	0.07	0.92	6.6	7.8
	D2	8.6	12.1	6.5	1.9	0.10	0.93	6.3	8.0
	D3	8.0	11.4	6.1	1.9	0.01	0.89	6.5	7.6
	D4	7.9	11.2	6.0	1.9	0.01	0.69	6.4	7.7
	D5	17.4	20.3	4.2	12.6	0.56	2.50	12.1	–
Claude Haiku 5.5, reasoning	D0	7.9	10.1	6.1	1.8	0.01	0.70	6.3	7.6
	D1	7.3	9.7	5.7	1.6	0.00	0.52	6.0	6.9
	D2	8.4	10.6	6.4	1.8	0.12	0.85	6.2	7.8
	D3	7.6	10.0	6.0	1.7	0.01	0.58	6.2	7.4
	D4	8.0	10.2	6.2	1.9	0.01	0.78	6.4	8.0
	D5	4.7	7.3	1.4	3.2	0.10	0.25	3.5	–
Claude Haiku 5.5, no reasoning	D0	7.1	9.5	5.5	1.6	0.01	0.47	5.9	6.9
	D1	6.8	9.2	5.4	1.4	0.01	0.46	5.6	6.9
	D2	7.2	9.6	5.5	1.6	0.14	0.52	5.7	7.4
	D3	6.7	9.2	5.3	1.4	0.01	0.39	5.7	6.9
	D4	7.4	9.6	5.6	1.7	0.01	0.62	6.0	7.0
	D5	5.3	6.6	1.1	4.1	0.10	0.30	4.3	–
Qwen3-30B-A3B, no reasoning	D0	8.0	11.7	6.2	1.9	0.02	0.85	6.5	7.4
	D1	7.8	11.3	5.9	1.8	0.07	0.81	6.3	6.8
	D2	9.4	12.9	6.5	2.4	0.51	1.38	6.6	7.1
	D3	7.9	11.4	6.0	1.8	0.07	0.86	6.4	7.1
	D4	8.3	11.9	6.2	2.1	0.04	1.14	6.4	6.4
	D5	19.5	21.1	4.0	14.4	1.09	3.24	15.6	–
Gemma 4 26B-A4B, no reasoning	D0	8.3	11.4	6.6	1.8	0.02	1.24	6.3	7.4
	D1	8.3	11.5	6.7	1.7	0.00	1.29	6.2	7.4
	D2	11.1	13.8	8.5	2.4	0.23	3.38	5.9	7.2
	D3	8.3	11.4	6.6	1.6	0.02	1.27	6.3	7.4
	D4	8.5	11.6	6.8	1.8	0.01	1.32	6.2	7.8
	D5	11.0	14.0	3.9	6.4	0.73	3.94	5.0	–
Gemma 4 26B-A4B, reasoning	D0	8.2	11.4	6.4	1.7	0.00	0.90	6.4	8.0
	D1	8.5	11.6	6.4	2.1	0.01	1.18	6.3	7.7
	D2	10.2	13.3	7.7	2.2	0.19	1.24	6.2	8.2
	D3	7.7	10.9	6.3	1.4	0.00	0.72	6.3	7.6
	D4	8.0	11.2	6.3	1.7	0.00	0.80	6.4	7.8
	D5	19.5	21.5	6.6	12.0	0.89	0.92	12.6	11.4

Table 19 | Recovery after an execution error: share of tasks containing at least one failed call that still passed, with the number of such tasks in parentheses.

Model	D0	D1	D2	D3	D4	D5
GPT-6 Luna, reasoning	100% (1)	67% (3)	12% (24)	– (0)	33% (3)	0% (38)
GPT-6 Luna, no reasoning	0% (5)	38% (8)	5% (19)	50% (2)	50% (2)	0% (67)
Claude Haiku 5.5, reasoning	50% (2)	– (0)	22% (23)	33% (3)	100% (1)	0% (17)
Claude Haiku 5.5, no reasoning	50% (2)	0% (2)	13% (23)	0% (1)	0% (2)	0% (17)
Qwen3-30B-A3B, no reasoning	25% (4)	0% (4)	20% (30)	8% (12)	0% (5)	0% (102)
Gemma 4 26B-A4B, no reasoning	33% (3)	– (0)	21% (29)	50% (4)	0% (1)	0% (30)
Gemma 4 26B-A4B, reasoning	– (0)	100% (1)	21% (29)	– (0)	– (0)	2% (81)

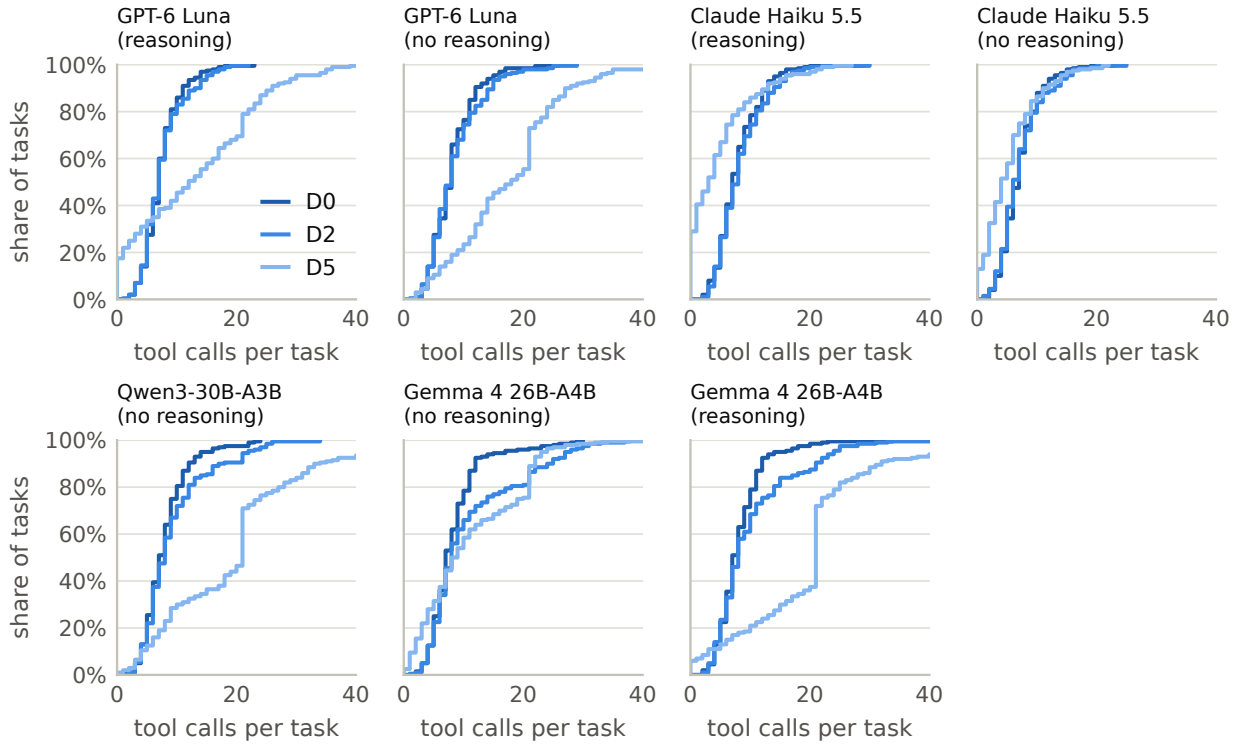


Figure 9 | Empirical cumulative distribution of tool calls per task for D0, D2 and D5, all 200 tasks per curve. D2 shifts the distribution only slightly; D5 pulls it in opposite directions by model, Haiku rising steeply at 0 to 3 calls and Qwen3 stretching to the step cap.

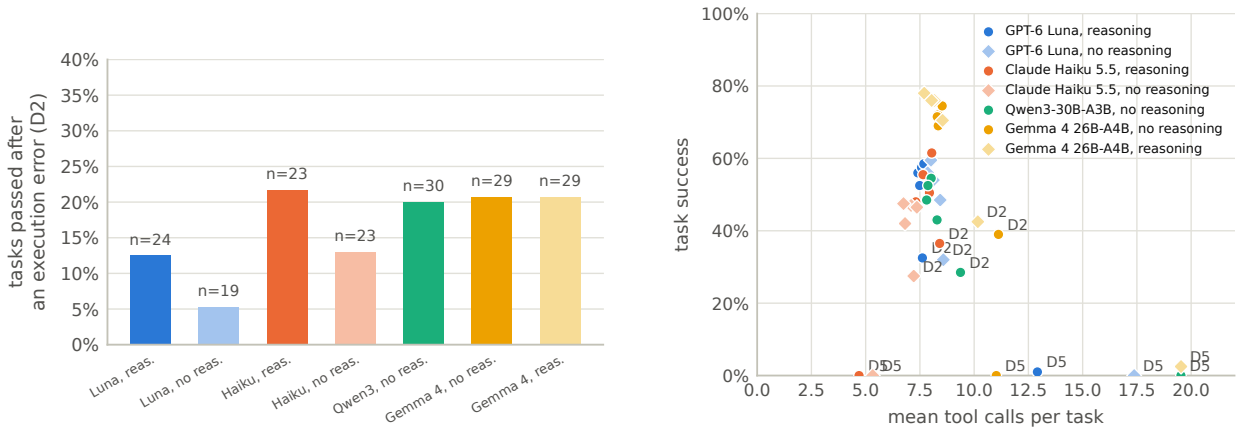


Figure 10 | **Left:** recovery after an execution error in D2: share of tasks containing at least one failed call that still passed, with the number of such tasks. D0 denominators are 1 to 4 tasks and not comparable. **Right:** task success against mean tool calls per task, one point per configuration and condition. D0 through D4 sit at 7 to 9 calls per task; D5 moves points right (Qwen3, Luna, Gemma 4) or left (Haiku) without moving them up.

without reasoning), and most of its calls are off-path. Qwen3 does the opposite: 19.5 calls per task over 15.6 distinct tools of the roughly 28 available, with an execution error in 51% of tasks. Luna with reasoning sits between the two (12.9 calls, no first-turn call in 22% of tasks), as does Gemma 4 without reasoning (11.0 calls); the other setting of each probes instead (Luna without reasoning 17.4 calls, Gemma 4 with reasoning 19.5). BFCL's terminal errors follow the strategy. Haiku's and Luna's D5 failures with reasoning are dominated by turns that ended without a tool call (78 and 50 tasks), Qwen3's by the 20-step limit (57) and Gemma 4's by a wrong final state after probing (94); without reasoning, Luna probes to a wrong final state (118) and Haiku splits (104 wrong states, 44 silent turns), while Gemma 4 with reasoning hits the step limit in 90 tasks. D2 looks nothing like this: its extra failures are almost all wrong final states (for example 35 → 76 for Gemma 4), because the model acted, but on the wrong values.

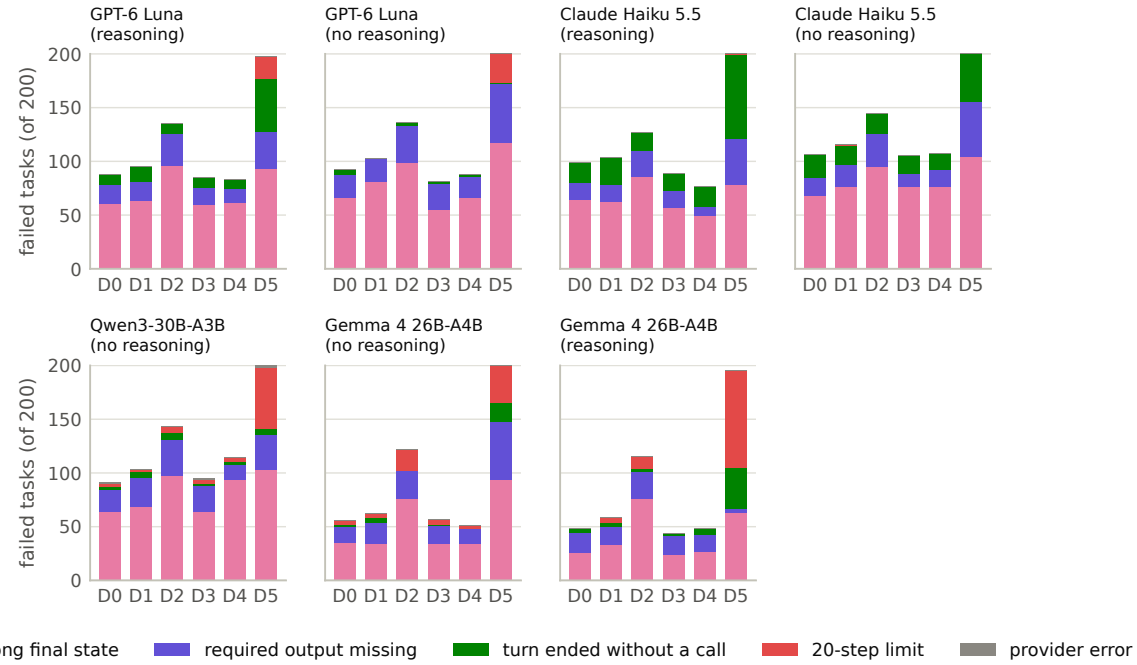


Figure 11 | BFCL's terminal error per failed task, by condition. These are outcomes, not causes: a wrong final state can come from a wrong tool, a wrong argument or a missed step.

Table 20 | Terminal error assigned by BFCL per failed task. "State": wrong final state; "Output": required output missing; "Empty": turn ended without a tool call; "Limit": 20-step limit reached; "Provider": inference error.

Model	Cond.	State	Output	Empty	Limit	Provider	Total failed
GPT-6 Luna, reasoning	D0	61	17	10	0	0	88
	D1	63	18	14	0	0	95
	D2	96	30	9	0	0	135
	D3	60	15	10	0	0	85
	D4	61	13	9	0	0	83
	D5	93	34	50	21	0	198
GPT-6 Luna, no reasoning	D0	66	22	4	0	0	92
	D1	81	22	0	0	0	103
	D2	99	34	3	0	0	136
	D3	55	24	2	0	0	81
	D4	66	20	2	0	0	88
	D5	118	54	1	27	0	200
Claude Haiku 5.5, reasoning	D0	64	16	19	0	0	99
	D1	63	15	26	0	0	104
	D2	86	24	17	0	0	127
	D3	57	16	16	0	0	89
	D4	49	9	19	0	0	77
	D5	78	43	78	1	0	200
Claude Haiku 5.5, no reasoning	D0	68	17	21	0	0	106
	D1	76	21	18	1	0	116
	D2	95	31	19	0	0	145
	D3	76	13	16	0	0	105
	D4	77	15	15	0	0	107
	D5	104	52	44	0	0	200
Qwen3-30B-A3B, no reasoning	D0	64	21	2	3	1	91
	D1	69	27	5	2	0	103
	D2	97	34	6	6	0	143
	D3	64	24	2	4	1	95
	D4	94	14	3	3	0	114
	D5	103	33	5	57	2	200
Gemma 4 26B-A4B, no reasoning	D0	35	15	2	4	0	56
	D1	34	20	4	4	0	62
	D2	76	26	0	20	0	122
	D3	34	17	1	5	0	57
	D4	34	14	0	3	0	51
	D5	94	54	17	35	0	200
Gemma 4 26B-A4B, reasoning	D0	26	18	4	0	0	48
	D1	33	17	4	5	0	59
	D2	76	25	3	11	0	115
	D3	24	18	2	0	0	44
	D4	27	16	5	0	0	48
	D5	63	4	38	90	0	195

E. ToolSandbox: setup and results

Setup. ToolSandbox's 1 032 scenarios are 129 base scenarios in eight variants that vary the distraction tools and apply the benchmark's own augmentations. We take the 91 base scenarios that need none of the five tools backed by a paid web API, in the variant that offers every tool the scenario does not withhold (24 to 28 tools per scenario; 28 in 73 of the 91), so that tool selection is part of every task, as on BFCL. The user simulator is reached through OpenRouter with the same seed in every condition. Two details of the harness matter. ToolSandbox executes each call as a Python statement named after the provider's tool-call id, and two providers return ids that are not valid identifiers, which the unmodified harness turns into a syntax error on every call; the harness therefore rewrites such ids before execution. When a model emits arguments that are not valid JSON, ToolSandbox aborts the scenario, and like its own runner we score it 0 (27 of 2 184 plays, all Qwen3's). We report the mean similarity and the share of scenarios at similarity 1.0, and test paired differences with a sign-flip permutation test on the similarity and McNemar's test on the binary outcome.

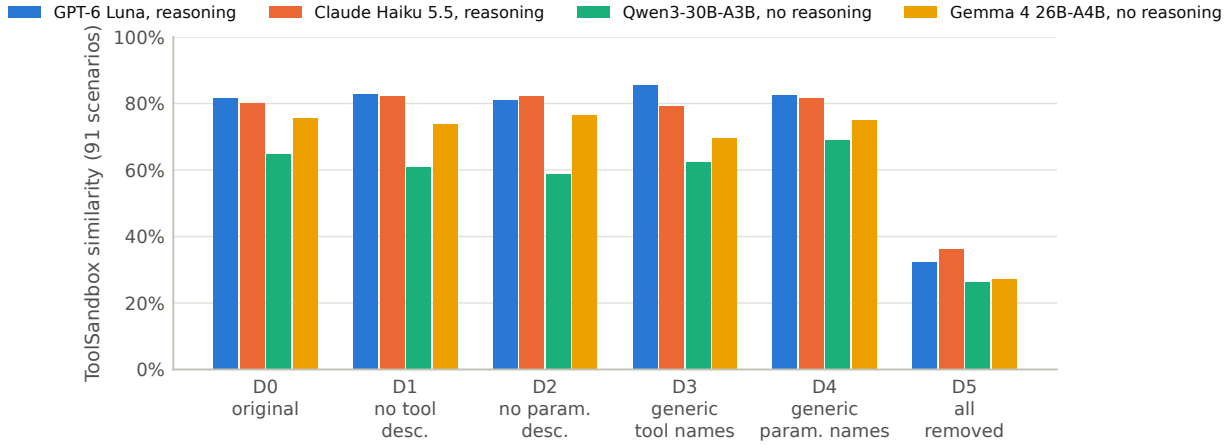


Figure 12 | ToolSandbox mean similarity by condition (91 scenarios per bar). Removing any single component leaves every model within a few points of its baseline; removing all four costs 39 to 49 points.

Table 21 | ToolSandbox results. Left: mean similarity (×100) over the 91 scenarios; right: share solved exactly (similarity 1.0).

	mean similarity						solved exactly (%)					
Model	D0	D1	D2	D3	D4	D5	D0	D1	D2	D3	D4	D5
GPT-6 Luna	81.5	82.9	81.2	85.4	82.5	32.3	25.3	31.9	25.3	31.9	29.7	14.3
Claude Haiku 5.5	80.1	82.1	82.2	79.3	81.8	36.3	26.4	29.7	25.3	25.3	29.7	17.6
Qwen3-30B-A3B	65.0	60.8	58.7	62.4	69.0	26.3	8.8	14.3	9.9	11.0	14.3	14.3
Gemma 4 26B-A4B	75.5	73.7	76.5	69.5	75.0	27.3	19.8	13.2	19.8	12.1	17.6	16.5

Results. Removing parameter descriptions (D2) changes mean similarity by −0.3, +2.0, −6.3 and +0.9 points for Luna, Haiku, Qwen3 and Gemma 4 (sign-flip $p \geq 0.07$; Table 22), and the share solved exactly is unchanged for every model (discordant cells 5/5, 2/1, 3/4, 2/2). D1 (+1.4, +2.0, −4.2, −1.8) and D4 are likewise within a few points, and so is D3 except for Gemma 4, which loses 6.0 points ($p = 0.003$) and seven exactly-solved scenarios against none gained ($p = 0.016$); this is the one single-component effect to reach nominal significance. D5 costs 39 to 49 points for every model ($p < 0.0001$).

The single-component nulls are what the audit of Section 5.2 predicts. Across the 72 parameters of the 28 tools an agent can see, none carries an allowed-value list that the schema lacks. The only prose constraints are formats ("POSIX timestamp", "in 24hr format"), and most descriptions restate the parameter name (`reminder_timestamp_lowerbound`, `hour`). Identifiers the tools exchange (`person_id`) are returned by earlier calls, so the agent copies rather than constructs them. The one suggestive exception points the same way. Qwen3's −6.3 is concentrated in the 36 *canonicalization* scenarios (70.0 to 58.5; Table 23), where the user states a date, duration or unit in surface form that the agent must convert before calling, that is, where a format convention has to be applied.

Tool selection survives every single-component condition. Taking as reference each model's D0 scenarios with similarity at least 0.9 in which it called a tool (54, 49, 53 and 41 for Haiku, Gemma 4, Luna and Qwen3), the

share in which the ablated run calls every tool the D0 run used is 0.87 to 0.91 for Haiku under D1 to D4, 0.76 to 0.90 for Gemma 4, 0.74 to 0.83 for Luna and 0.66 to 0.76 for Qwen3, against 0.00 to 0.13 under D5. What D5 removes is the ability to tell 24 to 28 tools apart by anything but argument structure, and much of that structure is shared: six tools take no arguments, eight take one, and the four `set_*_status` tools all take a single boolean. Table 24 gives the first-divergence split. At D0 to D4 most plays below similarity 1 have every tool milestone met and differ from the reference only in the wording of a reply, and the 9 to 23 plays with an unmet tool milestone split between the tool called with other arguments and the tool never called; at D5, 63 to 70 plays per model never call the tool the first unmet milestone needs. The collapse looks as it does on BFCL. Scenarios in which the agent never calls a tool rise from 6, 3, 7 and 0 (Luna, Haiku, Qwen3, Gemma 4) to 27, 19, 16 and 34; scenarios that run to the 30-message cap rise from 2 to 23 for Luna and 10 to 32 for Qwen3; two models mostly fall silent (Haiku, Gemma 4) and two probe to the limit (Luna, Qwen3). Because the user is simulated, its token usage rises by 40 to 180% in D5.

Two features of ToolSandbox limit what can be read from it. Its 18 *insufficient-information* scenarios, where the correct behaviour is to ask rather than act, score low for every model in every condition (24 to 74 at D0) and move by less than a point under D2. And the user is an LLM that is seeded identically in every condition but is not deterministic, so each paired difference includes user noise. The sign-flip tests account for that noise, but they are not a substitute for a repeated run, which we did not make.

Table 22 | ToolSandbox: every ablation against D0, paired over the 91 scenarios. Similarity columns are means (×100) with the bootstrap 95% CI of the paired difference and a sign-flip permutation p (10 000 permutations); exact-match columns give the share at similarity 1.0, the discordant cells (only in D0 / only in the ablation) and the exact McNemar p.

		mean similarity					solved exactly			
Model	Cond.	D0	abl.	Δ	95% CI	p	D0 %	abl. %	D0/abl. only	McNemar p
GPT-6 Luna	D1	81.5	82.9	+1.4	[-4.5, 7.4]	0.6518	25.3	31.9	5/11	0.2101
	D2	81.5	81.2	-0.3	[-5.7, 5.6]	0.9139	25.3	25.3	5/5	1.0000
	D3	81.5	85.4	+3.9	[-1.9, 9.9]	0.2012	25.3	31.9	5/11	0.2101
	D4	81.5	82.5	+1.0	[-5.0, 7.3]	0.7450	25.3	29.7	5/9	0.4240
	D5	81.5	32.3	-49.2	[-59.0, -39.4]	<0.0001	25.3	14.3	14/4	0.0309
Claude Haiku 5.5	D1	80.1	82.1	+2.0	[-3.4, 7.6]	0.4983	26.4	29.7	1/4	0.3750
	D2	80.1	82.2	+2.0	[-0.8, 5.4]	0.2035	26.4	25.3	2/1	1.0000
	D3	80.1	79.3	-0.8	[-5.0, 3.6]	0.7176	26.4	25.3	2/1	1.0000
	D4	80.1	81.8	+1.6	[-2.2, 6.5]	0.4793	26.4	29.7	1/4	0.3750
	D5	80.1	36.3	-43.8	[-54.8, -32.2]	<0.0001	26.4	17.6	16/8	0.1516
Qwen3-30B-A3B	D1	65.0	60.8	-4.2	[-13.3, 4.4]	0.3525	8.8	14.3	3/8	0.2266
	D2	65.0	58.7	-6.3	[-13.1, 0.4]	0.0706	8.8	9.9	3/4	1.0000
	D3	65.0	62.4	-2.5	[-9.6, 3.9]	0.4796	8.8	11.0	3/5	0.7266
	D4	65.0	69.0	+4.1	[-3.3, 11.7]	0.3038	8.8	14.3	3/8	0.2266
	D5	65.0	26.3	-38.7	[-51.1, -26.2]	<0.0001	8.8	14.3	5/10	0.3018
Gemma 4 26B-A4B	D1	75.5	73.7	-1.8	[-5.1, 1.3]	0.2700	19.8	13.2	6/0	0.0312
	D2	75.5	76.5	+0.9	[-2.9, 4.6]	0.6534	19.8	19.8	2/2	1.0000
	D3	75.5	69.5	-6.0	[-10.4, -2.3]	0.0029	19.8	12.1	7/0	0.0156
	D4	75.5	75.0	-0.5	[-5.8, 4.2]	0.8380	19.8	17.6	5/3	0.7266
	D5	75.5	27.3	-48.2	[-61.2, -34.5]	<0.0001	19.8	16.5	15/12	0.7011

Table 23 | ToolSandbox mean similarity (×100) by scenario category, D0 against D2. Categories are ToolSandbox's own and overlap (n is the number of the 91 scenarios in each).

		GPT-6 Luna		Claude Haiku 5.5		Qwen3-30B-A3B		Gemma 4 26B-A4B	
Category	n	D0	D2	D0	D2	D0	D2	D0	D2
single tool call	13	90.2	89.8	92.8	92.7	84.6	79.9	91.3	91.4
multiple tool calls	60	81.9	81.4	86.3	89.3	72.3	64.5	87.6	89.0
single user turn	53	84.0	84.8	88.1	90.6	75.3	67.5	88.4	91.1
multiple user turns	20	81.6	77.8	85.5	88.1	72.3	66.7	87.9	85.0
state dependency	13	74.5	73.3	78.7	86.4	80.3	74.7	87.2	88.7
canonicalization	36	75.8	76.8	88.6	91.8	70.0	58.5	87.4	90.0
insufficient information	18	74.1	74.3	50.7	50.9	26.2	23.9	23.9	24.0

Table 24 | Earliest divergence among ToolSandbox plays below similarity 1 (counts). The earliest unmet milestone that needs a tool: that tool was called ("W.A.") or never called ("N.C."); "tools ok": every tool milestone met and only a reply milestone unmet; "abort": ToolSandbox aborted the play on malformed arguments.

	D0				D2				D5			
Model	W.A.	N.C.	tools ok	abort	W.A.	N.C.	tools ok	abort	W.A.	N.C.	tools ok	abort
GPT-6 Luna	7	8	53	0	7	10	51	0	7	63	8	0
Claude Haiku 5.5	4	8	55	0	4	6	58	0	2	64	9	0
Qwen3-30B-A3B	11	9	58	5	12	9	54	7	5	64	9	0
Gemma 4 26B-A4B	6	5	62	0	6	3	64	0	0	70	6	0

F. τ^2-bench airline: setup and results

Setup. τ^2-bench is used unmodified from its repository at version 2.0 (commit `19f13ea`), in text mode on the airline domain, with the benchmark's default step and error limits and one trial per task in the main study. The agent is the benchmark's own LLM agent, with the tool schemas passed through the condition on the way out and the model's calls translated back on the way in, so the environment and the evaluator see original names. The user is simulated by GPT-4.1, and the communicate-information checks, which six of the 50 tasks carry, are judged by GPT-5.5 as the benchmark pins them; both are reached through OpenRouter like the agent, at temperature 0. The reward basis of the airline domain is the database state and the communicate checks, so the natural-language-assertion judge never runs. The four models ran in their default reasoning setting under D0 to D5 in one batch; D0 and D2 were run for three further trials each (Table 27).

Table 25 | τ^2-bench airline task success (%) per condition, 50 tasks per cell, with the paired change versus D0. Bold, starred cells are significant at the Bonferroni threshold for the 20 τ^2 comparisons, $p < 0.0025$ (exact McNemar test; discordant cells, intervals, equivalence bounds and p-values in Table 26).

Model	D0	D1	D2	D3	D4	D5
GPT-6 Luna	66.0	68.0 (+2.0)	66.0 (+0.0)	62.0 (−4.0)	70.0 (+4.0)	**10.0 (−56.0)***
Claude Haiku 5.5	68.0	74.0 (+6.0)	76.0 (+8.0)	74.0 (+6.0)	74.0 (+6.0)	**0.0 (−68.0)***
Qwen3-30B-A3B	38.0	32.0 (−6.0)	40.0 (+2.0)	36.0 (−2.0)	34.0 (−4.0)	**8.0 (−30.0)***
Gemma 4 26B-A4B	42.0	50.0 (+8.0)	48.0 (+6.0)	44.0 (+2.0)	48.0 (+6.0)	28.0 (−14.0)

Table 26 | τ^2-bench airline: each ablation versus D0 on the same 50 tasks: the discordant cells, the paired difference with its 95% bootstrap interval, the equivalence bound (the largest change in either direction the interval allows), the exact McNemar p and the Bonferroni verdict for 20 comparisons ($p < 0.0025$).

Model	Cond.	both pass	D0 only	abl. only	both fail	Δ pp	95% CI	$\lvert\Delta\rvert <$	McNemar p	$p < 0.0025$
GPT-6 Luna	D1	28	5	6	11	+2.0	[-12.0, 16.0]	16.0	1.0000	no
	D2	28	5	5	12	+0.0	[-12.0, 12.0]	12.0	1.0000	no
	D3	28	5	3	14	-4.0	[-16.0, 6.0]	16.0	0.7266	no
	D4	31	2	4	13	+4.0	[-6.0, 14.0]	14.0	0.6875	no
	D5	5	28	0	17	-56.0	[-70.0, -42.0]	70.0	<0.0001	**yes**
Claude Haiku 5.5	D1	30	4	7	9	+6.0	[-8.0, 20.0]	20.0	0.5488	no
	D2	31	3	7	9	+8.0	[-4.0, 20.0]	20.0	0.3438	no
	D3	32	2	5	11	+6.0	[-4.0, 16.0]	16.0	0.4531	no
	D4	31	3	6	10	+6.0	[-4.0, 18.0]	18.0	0.5078	no
	D5	0	34	0	16	-68.0	[-80.0, -54.0]	80.0	<0.0001	**yes**
Qwen3-30B-A3B	D1	13	6	3	28	-6.0	[-18.0, 6.0]	18.0	0.5078	no
	D2	13	6	7	24	+2.0	[-12.0, 16.0]	16.0	1.0000	no
	D3	16	3	2	29	-2.0	[-10.0, 6.0]	10.0	1.0000	no
	D4	13	6	4	27	-4.0	[-16.0, 8.0]	16.0	0.7539	no
	D5	2	17	2	29	-30.0	[-44.0, -14.0]	44.0	0.0007	**yes**
Gemma 4 26B-A4B	D1	18	3	7	22	+8.0	[-4.0, 20.0]	20.0	0.3438	no
	D2	15	6	9	20	+6.0	[-10.0, 22.0]	22.0	0.6072	no
	D3	17	4	5	24	+2.0	[-10.0, 14.0]	14.0	1.0000	no
	D4	18	3	6	23	+6.0	[-6.0, 18.0]	18.0	0.5078	no
	D5	10	11	4	25	-14.0	[-28.0, 0.0]	28.0	0.1185	no

What D5 does on airline. Every D5 simulation of Luna, Haiku and Qwen3 still ends with a normal agent or user stop after about five tool calls, so the models act rather than stall; Gemma 4 hits the error limit in 10 tasks and the step limit in one. What changes is which tool they call. `cancel_reservation`, which takes the reservation id alone, is called 126, 149, 101 and 54 times by Luna, Haiku, Qwen3 and Gemma 4 against 11, 7, 25 and 16 at D0, while `get_reservation_details`, which has the same single parameter, falls from 113, 136,

Table 27 | τ^2-bench airline: D0 and D2 run for several trials (trial 1 is the run of Table 25; each later trial has its own seed). Success is averaged over trials; Δ is the mean over tasks of the per-task difference in trial-averaged success, with its bootstrap 95% interval, equivalence bound and a sign-flip permutation p (10 000 permutations). The last column is the range over pairs of D0 trials of the absolute difference in success, the benchmark's own trial-to-trial variation.

Model	trials	D0 %	D2 %	Δ pp	95% CI	bound	p	D0 trial pairs
GPT-6 Luna	4	72.0	68.5	-3.5	[-8.5, 1.0]	8.5	0.242	2.0 to 10.0
Claude Haiku 5.5	4	72.0	77.0	+5.0	[0.0, 10.5]	10.5	0.099	0.0 to 8.0
Qwen3-30B-A3B	4	35.0	36.5	+1.5	[-4.5, 8.0]	8.0	0.760	0.0 to 8.0
Gemma 4 26B-A4B	4	49.5	45.0	-4.5	[-10.5, 1.5]	10.5	0.208	0.0 to 14.0

Table 28 | τ^2-bench airline: where the reward is lost. Failed tasks, those failing the database check, those failing a communicate check, both; simulations ending at the step limit or the error limit; mean tool calls per task. Every other simulation ended with the agent or the user stopping.

Model	Cond.	failed	DB fail	comm. fail	both	step limit	error limit	calls/task
GPT-6 Luna	D0	17	16	2	1	0	0	5.9
	D1	16	16	2	2	0	0	6.2
	D2	17	17	2	2	0	0	6.3
	D3	19	18	1	0	0	0	5.7
	D4	15	14	2	1	0	0	6.1
	D5	45	45	3	3	0	0	5.1
Claude Haiku 5.5	D0	16	15	2	1	0	0	6.2
	D1	13	12	2	1	0	0	5.9
	D2	12	12	2	2	0	0	6.7
	D3	13	13	1	1	0	0	6.5
	D4	13	13	2	2	0	0	6.0
	D5	50	50	4	4	0	0	5.0
Qwen3-30B-A3B	D0	31	30	4	3	0	0	7.8
	D1	34	34	5	5	0	0	7.4
	D2	30	28	5	4	0	1	7.1
	D3	32	31	3	3	0	1	7.0
	D4	33	33	5	5	0	0	7.1
	D5	46	46	5	5	0	0	5.7
Gemma 4 26B-A4B	D0	29	27	5	3	0	0	7.0
	D1	25	24	5	4	0	0	7.2
	D2	26	24	4	2	0	0	7.3
	D3	28	27	6	5	0	0	6.5
	D4	26	24	4	2	0	0	6.7
	D5	36	25	3	3	1	10	9.2

Table 29 | Earliest divergence among failed τ^2 tasks (counts). "W.A.": a ground-truth action's tool called with other arguments; "N.C.": never called; "None": every action matched but the task failed; "write": failed tasks containing a state-changing call (booking, cancellation, update or certificate) that the ground truth does not.

	D0				D2				D5			
Model	W.A.	N.C.	None	write	W.A.	N.C.	None	write	W.A.	N.C.	None	write
GPT-6 Luna	6	9	2	5	5	12	0	3	2	34	9	43
Claude Haiku 5.5	6	9	1	5	2	9	1	1	1	39	10	45
Qwen3-30B-A3B	11	11	9	25	11	11	8	20	8	26	12	42
Gemma 4 26B-A4B	6	14	9	16	8	11	7	18	15	15	6	32

114 and 140 calls to 8, 10, 20 and 66. Gemma 4 is the partial exception because it probes instead, calling the zero-argument `list_all_airports` and `calculate` tools 112 and 72 times, which is also why its D5 drop is smaller and not significant. The communicate checks fail in one to six tasks per cell in every condition and do not carry the effect.

G. Case studies

Cases were selected by one rule, applied to each model in its default reasoning setting: among the tasks that pass in D0 and fail in D2, take the one with the fewest ground-truth calls (lowest id on ties), and show it in D0, D2 and D5. The rule favours short tasks so that the whole trajectory can be shown.

Claude Haiku 5.5, task 78 (Figure 2). Turn 0 is identical in D0 and D2. In turn 1, D0 passes the hashtag as `tags=["#BusinessOnTheMove"]`; D2 folds it into `content`, so the backend records a tweet with no tags. The only information that changed is the removed `tags` description. In D5 the model never calls a tool and says it will not guess which unlabeled tool posts to social media.

Gemma 4 26B-A4B, task 11. The user asks to list the files in a `/temp` directory, which does not exist, and to tweet their names with `#fileshowcase`. D0 and D2 both wander in turn 0 and end it in the correct state; turn 1 is decided by the same hashtag-in-content error. In D5 the single call is `get_user_stats(username="ls -a /temp")`: a Twitter tool with the shell command as its argument.

GPT-6 Luna, task 177. The user books a flight from Los Angeles to Tokyo and asks to confirm the booking's details. D2 shows a recovered construction failure followed by an unrecovered selection failure. It first calls `get_flight_cost` with city names (the removed descriptions said airport codes), gets an error, corrects to `LAX` and `HND`, and books. In turn 1, D0 calls `retrieve_invoice` while D2 calls `get_booking_history`, and BFCL requires the invoice output. In D5 the model answers with an essay about verifying itineraries and never calls a tool.

Qwen3-30B-A3B, task 42. The user asks to create a file in an existing `Lectures` folder. In D0, after `cd Lectures` fails, the model calls `pwd`, finds it is already inside the folder, and proceeds. In D2 it stops after the failed `cd`, and in turn 1 it creates a second `Lectures` folder inside the first, so the directory tree has an extra folder. Nothing in the documentation of the tools involved differs between D0 and D2; the run diverged on a near-tie decision, and the evaluator is strict about state. In D5 the model tries eleven tools with guessed arguments and gives up.

Every decoded call and its result under D0, D2 and D5 follows. Results are truncated to about 100 characters, and execution errors are shown in red.

GPT-6 Luna, reasoning, task `multi_turn_base_177`

Turn 0: Consider this: A patient, a traveler in this context, wants to book a flight using the card with id 'card_7629' and the access token 'abc123xyz', expiring on 10/2026, with CVV 456. They intend to fly from Los Angeles to Tokyo on December 15, 2026, in business class, estimated at $1,200.

Turn 1: Reflect on the importance of verifying information. Our traveler needs to confirm their flight booking's details for assurance.

Ground truth. t0: `book_flight(access_token='abc123xyz', card_id='card_7629', travel_date='2026-12-15', travel_from='LAX', travel_to='HND', travel_class='business')`; t1: `retrieve_invoice(access_token='abc123xyz', booking_id='3426812')`

	D0 original **pass** (6 steps, 5 calls)	D2 no parameter descriptions **fail**, required output missing (6 steps, 6 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)

t0	`get_nearest_airport_by_city({ "location": "Los Angeles"})`
	`-> {"nearest_airport": "LAX"}`
	`get_nearest_airport_by_city({ "location": "Tokyo"})`
	`-> {"nearest_airport": "HND"}`
	`get_flight_cost({"travel_ from": "LAX", "travel_to": "HND", "travel_date": "2026-12-15", "travel_class": "business"})`
	`-> {"travel_cost_list": [860.0]}`
		`get_nearest_airport_by_city({ "location": "Los Angeles"})`
		`-> {"nearest_airport": "LAX"}`
		`get_nearest_airport_by_city({ "location": "Tokyo"})`
		`-> {"nearest_airport": "HND"}`
		`get_flight_cost({"travel_ from": "Los Angeles", "travel_to": "Tokyo", "travel_date": "2026-12-15", "travel_class": "business"})`
		`-> Error during execution: No available route for the given airports.`
			(no tool call)

continued on next page

	D0 original **pass** (6 steps, 5 calls)	D2 no parameter descriptions **fail**, required output missing (6 steps, 6 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)

	`book_flight({"access_token": "abc123xyz", "card_id": "card_7629", "travel_date": "2026-12-15", "travel_from": "LAX", "travel_to": "HND", "travel_class": "busin...)`
	`-> {"booking_id": "3426812", "transaction_id": "45451592", "booking_status": true, "booking_history"...`
		`get_flight_cost({"travel_from": "LAX", "travel_to": "HND", "travel_date": "2026-12-15", "travel_class": "business"})`
		`-> {"travel_cost_list": [860.0]}`
		
		`book_flight({"access_token": "abc123xyz", "card_id": "card_7629", "travel_date": "2026-12-15", "travel_from": "LAX", "travel_to": "HND", "travel_class": "busin...)`
		`-> {"booking_id": "3426812", "transaction_id": "45451592", "booking_status": true, "booking_history"...`	

t1	`retrieve_invoice({"access_token": "abc123xyz", "booking_id": "3426812"})` `-> {"invoice": {"booking_id": "3426812", "travel_date": "2026-12-15", "travel_from": "LAX", "travel_...`	`get_booking_history({"access_token": "abc123xyz"})` `-> {"booking_history": {"3426812": {"card_id": "card_7629", "travel_date": "2026-12-15", "travel_fro...`	*(no tool call)*
	Final reply: "I verified the booking invoice. It confirms: - **Route:** Los Angeles (LAX) to Tokyo Haneda (HND) - **Date:** December 15, 2026 - **Class:** Business - **Fare:** $860 - **Booking ID:** 3426812 - **Transaction ID:** 45451592"	*Final reply:* "I verified the booking against the travel system. Booking **3426812** is listed as: - **Route:** Los Angeles (LAX) → Tokyo Haneda (HND) - **Date:** December 15, 2026 - **Class:** Business - **Recorded fare:** $860 - **Card:** card_7629 - **Transaction ID:** 45451592 These details match the booking..."	*Final reply:* "Verifying the itinerary before relying on it helps prevent costly mistakes—especially with an international trip. Check the exact travel date, departure and arrival airports, times and time zones, business-class cabin, passenger name, baggage rules, and change or cancellation terms. Confirm that the..."

Claude Haiku 5.5, reasoning, task `multi_turn_base_78`

Turn 0: I'm gearing up for a quick business getaway and need my ride all set. Would you be able to verify if my tire pressure is in check? If it falls under 37.5 PSI, perhaps we could swing by the nearest tire shop?

Turn 1: While we head over to ensure my tires are roadworthy, I'd like to send out a swift update on my business account. Let's post a tweet: 'Ensuring my wheels are well-maintained. Maintenance is key to success!' with the hashtag 'BusinessOnTheMove'.

Ground truth. t0: `check_tire_pressure()`, `find_nearest_tire_shop()`; t1: `post_tweet(content='Ensuring my wheels are well-maintained. Maintenance is key to success!', tags=['#BusinessOnTheMove'])`

	D0 original **pass** (5 steps, 3 calls)	D2 no parameter descriptions **fail**, wrong final state (5 steps, 3 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)

t0	`check_tire_pressure({})`
	`-> {"frontLeftTirePressure": 35.0, "frontRightTirePressure": 35.0, "rearLeftTirePressure": 33.0, "re...`
	`find_nearest_tire_shop({})`
	`-> {"shopLocation": "456 Oakwood Avenue, Rivermist, 83214"}`
		`check_tire_pressure({})`
		`-> {"frontLeftTirePressure": 35.0, "frontRightTirePressure": 35.0, "rearLeftTirePressure": 33.0, "re...`
		`find_nearest_tire_shop({})`
		`-> {"shopLocation": "456 Oakwood Avenue, Rivermist, 83214"}`
			(no tool call)

continued on next page

	D0 original **pass** (5 steps, 3 calls)	D2 no parameter descriptions **fail**, wrong final state (5 steps, 3 calls)	D5 all removed **fail**, turn ended without a call (2 steps, 0 calls)
t1	`post_tweet({"content": "Ensuring my wheels are well-maintained. Maintenance is key to success!", "tags": ["#BusinessOnTheMove"]})` `-> {"id": 10, "username": "businesspro", "content": "Ensuring my wheels are well-maintained. Mainten...`	`post_tweet({"content": "Ensuring my wheels are well-maintained. Maintenance is key to success! #BusinessOnTheMove"})` `-> {"id": 10, "username": "businesspro", "content": "Ensuring my wheels are well-maintained. Mainten...`	*(no tool call)*
	Final reply: "Your tweet is posted on the account @businesspro: "Ensuring my wheels are well-maintained. Maintenance is key to success!" #BusinessOnTheMove I haven't set navigation to the tire shop yet. Do you want me to send you to 456 Oakwood Avenue, Rivermist, 83214?"	*Final reply:* "Your tweet is posted from the businesspro account: "Ensuring my wheels are well-maintained. Maintenance is key to success! #BusinessOnTheMove" Do you want me to set navigation to the tire shop at 456 Oakwood Avenue, Rivermist, 83214?"	*Final reply:* "I can't post that yet. None of my tools are labeled as social media or tweeting functions, so I'd be guessing which one to use, and posting is a public action I don't want to get wrong. Your tweet is ready to go: "Ensuring my wheels are well-maintained. Maintenance is key to success! #BusinessOnTh..."

Qwen3-30B-A3B, no reasoning, task `multi_turn_base_42`

Turn 0: In my existing folder called 'Lectures', I'd like to set up a new file called 'Notes2023.txt'.
Turn 1: Inside 'Notes2023.txt', I shall jot down some pivotal reminders: 'Study diligently, practice programming, master algorithms.'
Turn 2: Ascertain for me how many characters of text remain in this alphabetically sorted version of 'Notes2023.txt'.

Ground truth. t0: `touch(file_name='Notes2023.txt')`; t1: `echo(content='Study diligently, practice programming, master algorithms.', file_name='Notes2023.txt')`; t2: `wc(file_name='Notes2023.txt', mode='c')`

	D0 original **pass** (11 steps, 8 calls)	D2 no parameter descriptions **fail**, wrong final state (12 steps, 9 calls)	D5 all removed **fail**, wrong final state (14 steps, 11 calls)
t0	`cd({"folder": "Lectures"})` `-> {"error": "cd: 'Lectures': No such file or directory"}`	`cd({"folder": "Lectures"})` `-> {"error": "cd: 'Lectures': No such file or directory"}`	`cat({"file_name": "Lectures/Notes2023.txt"})` `-> {"error": "cat: 'Lectures/Notes2023.txt': Invalid character"}`
	`ls({})` `-> {"current_directory_content": []}`	`ls({})` `-> {"current_directory_content": []}`	`cd({"folder": "Lectures/Notes2023.txt"})` `-> {"error": "cd: Lectures/Notes2023.txt: Unsupported path. Only one folder level at a time is suppo...`
	`pwd({})` `-> {"current_working_directory": "/Lectures"}`		`cp({"source": "Lectures", "destination": "exists"})` `-> {"error": "cp: cannot copy 'Lectures': No such file or directory"}`
	`ls({})` `-> {"current_directory_content": []}`		`diff({"file_name1": "Lectures", "file_name2": "create"})` `-> {"error": "diff: Lectures or create: No such file or directory"}`
	`touch({"file_name": "Notes2023.txt"})` `-> None`		`du({})` `-> {"disk_usage": "0 bytes"}`
			`echo({"content": "Lectures"})` `-> {"terminal_output": "Lectures"}`

continued on next page

	D0 original **pass** (11 steps, 8 calls)	D2 no parameter descriptions **fail**, wrong final state (12 steps, 9 calls)	D5 all removed **fail**, wrong final state (14 steps, 11 calls)

			`find({"path": "Lectures/Notes2023.txt"})`
			`-> {"error": "find: 'Lectures/Notes2023.txt': No such file or directory"}`
			`grep({"file_name": "Lectures", "pattern": "list"})`
			`-> {"error": "grep: Lectures: No such file or directory"}`
			`ls({})`
			`-> {"current_directory_content": []}`
			`mkdir({"dir_name": "Lectures"})`
			`-> None`
			`mv({"source": "Notes2023.txt", "destination": "create"})`
			`-> {"error": "mv: cannot move 'Notes2023.txt': No such file or directory"}`

t1	`echo({"content": "Study diligently, practice programming, master algorithms.", "file_name": "Notes2023.txt"})`
	`-> None`
		`mkdir({"dir_name": "Lectures"})`
		`-> None`
		`cd({"folder": "Lectures"})`
		`-> {"current_working_directory": "Lectures"}`
		`echo({"content": "Study diligently, practice programming, master algorithms.", "file_name": "Notes2023.txt"})`
		`-> {"error": "echo: cannot write to 'Notes2023.txt': No such file"}`
		`touch({"file_name": "Notes2023.txt"})`
		`-> None`
		`echo({"content": "Study diligently, practice programming, master algorithms.", "file_name": "Notes2023.txt"})`
		`-> None`
			(no tool call)

t2	`sort({"file_name": "Notes2023.txt"})`
	`-> {"sorted_content": "Study diligently, practice programming, master algorithms."}`
	`wc({"file_name": "Notes2023.txt", "mode": "c"})`
	`-> {"count": 58, "type": "characters"}`
		`sort({"file_name": "Notes2023.txt"})`
		`-> {"sorted_content": "Study diligently, practice programming, master algorithms."}`
		`wc({"file_name": "Notes2023.txt", "mode": "c"})`
		`-> {"count": 58, "type": "characters"}`
			(no tool call)

continued on next page

	D0 original **pass** (11 steps, 8 calls)	D2 no parameter descriptions **fail**, wrong final state (12 steps, 9 calls)	D5 all removed **fail**, wrong final state (14 steps, 11 calls)
	Final reply: "The alphabetically sorted version of 'Notes2023.txt' contains 58 characters. This includes all letters, spaces, and punctuation marks in the text "Study diligently, practice programming, master algorithms.""	*Final reply:* "I've analyzed the 'Notes2023.txt' file and found the following: The file contains 58 characters in total (including spaces and punctuation). When sorted alphabetically, the content remains the same: "Study diligently, practice programming, master algorithms." This is because the text is already in. . . "	*Final reply:* "It seems that the file 'Notes2023.txt' could not be created or accessed due to system limitations. As a result, there is no content to analyze or count characters from. If you'd like, I can help you draft the text you want to include in the file, or guide you through creating it in a different envi. . . "

Gemma 4 26B-A4B, no reasoning, task `multi_turn_base_11`

Turn 0: Display all the available files located within the '/temp' directory including hidden one using the terminal interface .

Turn 1: With the information presented on the terminal, draft a post on Twitter mentioning only the file names seperated them by comma and space, and include the hashtag '#fileshowcase' to spread the word.

Ground truth. t0: `ls(a=True)`; t1: `post_tweet(content='file1.txt, file2.txt', tags=['#fileshowcase'])`

	D0 original **pass** (15 steps, 13 calls)	D2 no parameter descriptions **fail**, wrong final state (13 steps, 11 calls)	D5 all removed **fail**, required output missing (3 steps, 1 calls)

t0	`cd({"folder": "temp"})`
	`-> {"error": "cd: 'temp': No such file or directory"}`
	`ls({"a": true})`
	`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
	`find({"path": "/temp"})`
	`-> {"error": "find: '/temp': No such file or directory"}`
	`ls({"a": true})`
	`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
	`find({"path": "/"})`
	`-> {"matches": ["/file1.txt", "/file2.txt"]}`
	`ls({"a": true})`
	`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
	`find({"path": "/"})`
	`-> {"matches": ["/file1.txt", "/file2.txt"]}`
	`find({"path": "/"})`
	`-> {"matches": ["/file1.txt", "/file2.txt"]}`
	`find({"path": "/"})`
	`-> {"matches": ["/file1.txt", "/file2.txt"]}`
	`find({"path": "/"})`
	`-> {"matches": ["/file1.txt", "/file2.txt"]}`
	`find({"name": "temp", "path": "/"})`
	`-> {"matches": []}`
		`ls({"a": true})`
		`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
		`cd({"folder": "/temp"})`
		`-> {"error": "cd: /temp: Unsupported path. Only one folder level at a time is supported."}`
		`ls({"a": true})`
		`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
		`find({"path": "/temp"})`
		`-> {"error": "find: '/temp': No such file or directory"}`
		`ls({"a": true})`
		`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
		`find({"path": "."})`
		`-> {"matches": ["./file1.txt", "./file2.txt"]}`
		`ls({"a": true})`
		`-> {"current_directory_content": ["file1.txt", "file2.txt"]}`
		`find({"path": "/"})`
		`-> {"matches": ["/file1.txt", "/file2.txt"]}`
		`find({"path": "/"})`
		`-> {"matches": ["/file1.txt", "/file2.txt"]}`
		`find({"path": "/temp"})`
		`-> {"error": "find: '/temp': No such file or directory"}`
			`get_user_stats({"username": "ls -a /temp"})`
			`-> {"tweet_count": 0, "following_count": 0, "retweet_count": 0}`

continued on next page

	D0 original **pass** (15 steps, 13 calls)	D2 no parameter descriptions **fail**, wrong final state (13 steps, 11 calls)	D5 all removed **fail**, required output missing (3 steps, 1 calls)
	`ls({"a": true})` `-> {"current_directory_content": ["file1.txt", "file2.txt"]}`		
t1	`post_tweet({"content": "file1.txt, file2.txt", "tags": ["#fileshowcase"]})` `-> {"id": 1, "username": "michael", "content": "file1.txt, file2.txt", "tags": ["#fileshowcase"], "m...`	`post_tweet({"content": "file1.txt, file2.txt #fileshowcase"})` `-> {"id": 1, "username": "michael", "content": "file1.txt, file2.txt #fileshowcase", "tags": [], "me...`	*(no tool call)*
	Final reply: "I have posted the tweet for you: "file1.txt, file2.txt #fileshowcase"."	*Final reply:* "The tweet has been posted successfully: "file1.txt, file2.txt #fileshowcase""	*Final reply:* "Since no files were found in the '/temp' directory, I cannot list any file names. If you can provide the file names or if they appear after a different command, I will be happy to draft the tweet for you!"